MediHaven: Integrated Multimodal Clinical Intelligence and Sovereign Medical Vault Platform

submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology
in
Computer Engineering

T.E. Mini Project (Semester V)

by

Vidhi [Surname] UID: [UID]
Nirupam [Surname] UID: [UID]
Param [Surname] UID: [UID]

Group No.: **[Group No.]**

under the guidance of

[Guide name and designation]



Department of Computer Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri-West, Mumbai-400058
University of Mumbai

Academic Year 2026–27

Certificate

This is to certify that the project entitled "MediHaven: Integrated Multimodal Clinical Intelligence and Sovereign Medical Vault Platform" by Vidhi **[Surname]**, Nirupam **[Surname]** and Param **[Surname]** is submitted for the partial fulfillment of the T.E. Mini Project (Semester V) under the guidance of **[Guide name and designation]** for the award of the Degree of Bachelor of Technology in Computer Engineering from the University of Mumbai.

Certified by

[Guide name and designation]
Project Guide

[Head of Department name]
Head of Department

[Principal name]
Principal

Department of Computer Engineering
Bharatiya Vidya Bhavan's Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri (W), Mumbai-400058
[Month Year of submission]

Approval Certificate

This is to certify that the project entitled "MediHaven: Integrated Multimodal Clinical Intelligence and Sovereign Medical Vault Platform" by Vidhi [Surname], Nirupam [Surname] and Param [Surname] is approved for the partial fulfillment of the T.E. Mini Project (Semester V) towards obtaining the Degree of Bachelor of Technology in Computer Engineering from the University of Mumbai.

Project Guide

(signature)

Name:

Date:

External Examiner

(signature)

Name:

Date:

Head of Department

Principal

Seal of the Institute

Declaration

We wish to state that the work embodied in this report titled "MediHaven: Integrated Multimodal Clinical Intelligence and Sovereign Medical Vault Platform" forms our own contribution to the work carried out under the guidance of **[Guide name and designation]** at the Sardar Patel Institute of Technology. We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

We further declare that the implementation, test and evaluation results reported here are those actually obtained from the project repository, that features are described as implemented only where supporting evidence exists, and that MediHaven is an educational software prototype that has not been clinically validated.

Vidhi [Surname]	**Nirupam** [Surname]	**Param** [Surname]
UID: **[UID]**	UID: **[UID]**	UID: **[UID]**

Date: _____ Place: Mumbai

Acknowledgement

We express our sincere gratitude to our project guide, **[Guide name and designation]**, for guidance and feedback throughout the development of this project, and to the Head of the Department of Computer Engineering and the Principal of Sardar Patel Institute of Technology for providing the facilities and academic environment in which this work was carried out.

We thank the creators and maintainers of the MIMIC-IV Clinical Database Demo and PhysioNet for making de-identified critical-care data openly available for research and teaching, and the developers of the open-source software on which MediHaven is built, in particular Flask, scikit-learn, pandas, DuckDB and Chart.js.

Finally, we thank our families and classmates for their support and encouragement.

Vidhi **[Surname]** Nirupam **[Surname]** Param **[Surname]**

Abstract

Clinical deterioration is often recognised late, patients seldom control who sees their records, and published clinical machine-learning results are frequently hard to reproduce. MediHaven is an educational software prototype that addresses these issues together in a single Flask web application. Physicians see a triage dashboard in which a K-Means, decision-tree, k-nearest-neighbour and multilayer-perceptron ensemble classifies synthetic inpatients into four risk tiers, with every model's vote, a decision-tree rule path and similar historical cases. Patients keep a medical vault and share selected records through HMAC-signed, expiring, scope- and use-limited QR passes whose redemption is atomic and audited. An explicitly opt-in, loopback-only research mode rebuilds an in-hospital-mortality task from the MIMIC-IV Clinical Database Demo (prediction at ICU admission + 24 hours) and serves only hash-verified artifacts.

Built with Python, Flask, SQLite, scikit-learn, DuckDB and Chart.js, the system applies database design, concurrency control, algorithm analysis and software-engineering practice, and is tested by 354 automated tests, all passing. Under a load test of the production server, a risk prediction took 71.9 ms (median, one client), throughput saturated at about 15 predictions per second (CPU-bound), simple reads reached 268.8 requests per second at 16 clients, there were no errors, and memory stayed below 210 MB.

On the 107 eligible ICU stays (11 deaths), patient-grouped five-fold cross-validation gave pooled AUROCs of 0.74 (logistic regression), 0.73 (random forest), 0.77 (neural network) and 0.56 (smoothed decision tree), with 95 % intervals spanning roughly 0.35–0.91, and results changed markedly when four outcome-timestamp anomalies were removed. By contrast, the synthetic ensemble scored 1.00 on a 20-patient hold-out, which reflects the data generator rather than clinical skill. MediHaven therefore demonstrates an honest, reproducible engineering pipeline; it is not clinically validated and must not be used for patient care.

Keywords: clinical decision support, early warning, explainable machine learning, MIMIC-IV, grouped cross-validation, patient-controlled records, QR access, Flask.

Contents

Acknowledgement **iv**

Abstract **v**

List of Abbreviations and Symbols **xi**

1 Introduction and Problem Definition **1**

1.1 Project Background 1
1.2 Problem Statement 1
1.3 Motivation 1
1.4 Objectives 2
1.5 Scope of the Project 2
1.6 Proposed Solution 2
1.7 Novelty and Originality 3
1.8 Feasibility Analysis 3
1.9 Social Relevance and Practical Applications 3
1.10 Success Criteria and Key Performance Indicators 3
1.11 Organisation of the Report 3

2 Literature Review and Study of Existing Systems **5**

2.1 Study of Existing Systems 5
2.2 Review of Research Papers and Technical Resources 5
2.3 Comparative Analysis 7
2.4 Limitations of Existing Systems 7
2.5 Identified Engineering Gap 8
2.6 Proposed Improvements 8

3 Software Requirements Specification (SRS) **9**

3.1 Purpose and Scope of the SRS 9
3.2 Intended Users and Stakeholders 9
3.3 Overall Product Description 9
3.4 Functional Requirements 9
3.5 Non-Functional Requirements 10
3.6 Hardware and Software Requirements 11
3.7 Constraints and Assumptions 11
3.8 Use Cases and User Stories 11

4 System Architecture and Design **13**

4.1 Overall System Architecture 13
4.2 Component and Module Description 14

4.3 Architecture Selection and Justification . . . 14
4.4 Data Flow Diagrams . . . 14
4.5 UML Diagrams . . . 14
4.6 Database Design . . . 15
4.7 Security and Error Handling Design . . . 18

5 Algorithm Design and Data Structures 20
5.1 Identification of Critical Operations . . . 20
5.2 Algorithm Design . . . 20
5.3 Flowcharts . . . 21
5.4 Data Structure Selection . . . 21
5.5 Algorithm Selection and Justification . . . 22
5.6 Time Complexity Analysis . . . 23
5.7 Space Complexity Analysis . . . 23
5.8 Optimisation Opportunities . . . 23

6 Subject-Wise Engineering Concepts and APNI Leap Alignment 24
6.1 Software Engineering (CE202) . . . 24
6.2 Computer Networks . . . 24
6.3 Database Management Systems (CE204) . . . 24
6.4 Operating Systems (CE205) . . . 25
6.5 Data Structures (CE207) . . . 25
6.6 Design and Analysis of Algorithms (CE207) . . . 25
6.7 Applied Mathematics and Statistics (AS202) . . . 25
6.8 Cloud Computing . . . 25
6.9 Mapping of Subjects to Project Implementation . . . 25

7 Implementation and Module-Wise Coding 27
7.1 Development Environment . . . 27
7.2 Overall Implementation Strategy . . . 27
7.3 Module-Wise Implementation . . . 27
7.4 Front-End Implementation . . . 28
7.5 Back-End Implementation . . . 28
7.6 Database Implementation . . . 28
7.7 Communication and Concurrency Implementation . . . 29
7.8 Git Repository Setup . . . 29
7.9 Integration of Modules . . . 29
7.10 Implementation Challenges and Resolutions . . . 29

8 Scalability, Performance and Resource Management 31
8.1 Scalability Requirements . . . 31
8.2 Workload and Load Model . . . 31
8.3 Performance Measurement Setup . . . 31
8.4 Performance Results . . . 31
8.5 Resource Utilisation . . . 31
8.6 Concurrent Processing, Scheduling and Synchronisation . . . 32
8.7 Bottleneck Identification . . . 32

8.8 Optimisation and Improvement . . . 33
8.9 Cloud Deployment and Resource Utilisation . . . 33
8.10 Scalability Limitations . . . 33
9 Testing and Validation **34**
9.1 Testing Strategy . . . 34
9.2 Unit, Integration and Functional Testing . . . 34
9.3 Non-Functional, Boundary and Invalid-Input Testing . . . 34
9.4 Concurrency and Stress Testing . . . 35
9.5 Test Case Specification . . . 35
9.6 Defect Identification and Resolution . . . 36
9.7 Validation Against KPIs . . . 37
9.8 Limitations and Untested Scenarios . . . 37
10 Results, Demonstration and Discussion **38**
10.1 Implemented Features . . . 38
10.2 Screenshots and Demonstration Evidence . . . 38
10.3 Functional Results . . . 38
10.4 Performance Results . . . 39
10.5 Comparison with Existing Systems . . . 41
10.6 Achievement of Objectives . . . 42
10.7 Discussion of Findings . . . 42
10.8 Practical Applicability . . . 43
11 Conclusion and Future Scope **44**
11.1 Conclusion . . . 44
11.2 Achievement of Objectives . . . 44
11.3 Major Engineering Contributions . . . 44
11.4 Current Limitations . . . 44
11.5 Future Enhancements . . . 44
11.6 Lessons Learned . . . 45
Research Paper and Plagiarism Report **46**
References **47**
A Reproducibility and API Summary **49**
NIL AI Content Declaration **50**

List of Figures

3.1 Use-case diagram 11

4.1 Layered architecture of MediHaven 13
4.2 Context-level (level-0) data flow diagram 15
4.3 Level-1 data flow diagram 15
4.4 Class diagram of the model and service layer 16
4.5 Sequence diagram: redeeming a QR access pass 16
4.6 Activity diagram: synthetic risk prediction 17
4.7 State machine of a QR access pass 17
4.8 Deployment diagram: production (synthetic) and local research mode 18
4.9 Entity–relationship diagram (PK underlined, FK in italics; 1:N relationships) . . 18

5.1 Flowchart of the research pipeline (dataset v1.2.0, task v3, run v2.0.0) 22

8.1 Throughput and median latency against concurrency (same legend in both panels) 32

10.1 Physician portal on the synthetic cohort 38
10.2 Patient, administrator and scanner portals (synthetic data) 39
10.3 Research mode on the MIMIC-IV Demo (aggregate views only) 40
10.4 Pooled out-of-fold AUROC with 95% subject-bootstrap intervals (Table 10.2); dashed line: chance (0.5) 41

List of Tables

1.1 Key performance indicators and targets 4

2.1 Comparison of existing approaches with MediHaven 7

3.1 Functional requirements 9
3.2 Non-functional requirements 10

4.1 Major modules: purpose, inputs, outputs and dependencies 14

5.1 Theoretical complexity of the main algorithms 23

6.1 Mapping of subjects to project modules and evidence 26

7.1 Module-wise implementation summary 27

8.1 Measured latency and throughput (200 requests per cell, 0 errors in all 3,200 requests) 32

9.1 Representative test cases (actual result = observed in the run above) 35
9.2 Significant defects found and resolved 36
9.3 KPI targets (Table 1.1) against measured values 37

10.1 Research cohort flow (MIMIC-IV Clinical Database Demo 2.2) 41
10.2 Research baselines, primary analysis (107 stays, 11 deaths; evaluation status: limited) 41
10.3 Objectives and supporting evidence 42

List of Abbreviations and Symbols

API	Application Programming Interface	KPI	Key Performance Indicator
AP	Average Precision	LLM	Large Language Model
AUROC	Area Under the Receiver Operating Characteristic curve	MAP	Mean Arterial Pressure
BP	Blood Pressure (SBP systolic, DBP diastolic)	MIMIC	Medical Information Mart for Intensive Care
BUN	Blood Urea Nitrogen	MLP	Multilayer Perceptron
CI	Confidence Interval	OOF	Out-of-Fold (prediction)
CSP	Content Security Policy	OLS	Ordinary Least Squares
CV	Cross-Validation	PHR	Personal Health Record
DFD	Data Flow Diagram	QR	Quick Response (code)
EHR	Electronic Health Record	RBAC	Role-Based Access Control
ER	Entity–Relationship	REST	Representational State Transfer
FR / NFR	Functional / Non-Functional Requirement	SpO_2	Peripheral oxygen saturation
HMAC	Hash-based Message Authentication Code	SQL	Structured Query Language
HR	Heart Rate	SRS	Software Requirements Specification
ICU	Intensive Care Unit	TLS	Transport Layer Security
JWT	JSON Web Token	UML	Unified Modeling Language
KNN	k-Nearest Neighbours	WAL	Write-Ahead Logging (SQLite)
p	cohort death rate (prevalence)	$\hat{y}$	predicted probability of the positive class
n, d	number of samples, number of features	k	number of neighbours / clusters / folds

1 Introduction and Problem Definition

1.1 Project Background

Hospital wards generate a continuous stream of vital signs and laboratory results, yet deterioration is still often recognised late. Track-and-trigger scores such as the National Early Warning Score (NEWS) [1] and severity scores such as SAPS II [2] reduce physiology to a few thresholds, while machine-learning (ML) models trained on electronic health records (EHRs) report higher discrimination [3, 4]. At the same time, patients rarely control who sees their own records, and clinicians who receive a patient from another facility must often work without the history.

MediHaven is a web application, developed as a T.E. Mini Project (Semester V) in the Department of Computer Engineering, that brings three strands together in one teaching-scale system: (i) a physician triage dashboard that ranks synthetic inpatients with four classical ML models and explains each vote, (ii) a patient-controlled medical vault from which time-limited, scope-limited QR access passes can be shared, and (iii) an explicitly opt-in research mode that rebuilds a real mortality-prediction task from the openly available MIMIC-IV Clinical Database Demo [5, 6] and evaluates baselines with leakage-aware, patient-grouped cross-validation.

MediHaven is an **educational software prototype**. It is not a medical device, it has not been clinically validated, and none of its outputs may be used for diagnosis, treatment or other patient-care decisions.

1.2 Problem Statement

Build a single, secure, testable web platform that (a) demonstrates explainable multi-model risk stratification on realistic but synthetic inpatient data, (b) lets a patient grant a clinician revocable, auditable and scope-restricted access to selected records, and (c) shows, on genuine de-identified ICU data, how a mortality-risk pipeline must be built and evaluated honestly—without leakage, with uncertainty, and without exposing patient-level data beyond the researcher's machine.

Existing approaches make this difficult: published models rarely release reproducible cohorts [7], deployed proprietary scores can perform far worse than reported [8], and record-sharing systems are typically institution-centred rather than patient-controlled [9].

1.3 Motivation

The project gives three groups something concrete: students and teachers obtain an end-to-end example of clinical ML engineering (data pipeline, models, evaluation, serving, security and testing); clinicians and evaluators can see how model votes, rule paths and uncertainty are presented; and patients see a model of consent-based sharing with a visible access log.

The research mode in particular demonstrates the gap between an impressive demo accuracy on synthetic data and the much more modest, uncertain performance obtained on real data (Chapter 10).

1.4 Objectives

O1. Implement role-based web portals for physicians, patients and administrators over a REST API.
O2. Train and serve a four-model ensemble (K-Means, decision tree, KNN, MLP) that returns a risk tier, a score, per-model votes and an explanation for each synthetic patient.
O3. Implement a patient vault with HMAC-signed, expiring, scope- and use-limited QR passes, an atomic use quota and an immutable access log.
O4. Build a versioned MIMIC-IV Demo task (in-hospital mortality after the first 24 ICU hours) with explicit cohort, label and leakage rules, and evaluate baselines with patient-grouped cross-validation and bootstrap confidence intervals.
O5. Serve verified research artifacts in a loopback-only research mode with integrity checks.
O6. Protect secrets and data: strong-secret enforcement, CSP, rate limits, de-identified prompts to external AI, and no external AI in research mode.
O7. Verify the system with an automated test suite and measure server performance under load.

1.5 Scope of the Project

Included: the features above; a synthetic cohort of 100 patients; the MIMIC-IV Demo 2.2 (100 patients, 140 ICU stays); a production entry point (Waitress) and container recipe. **Excluded:** real hospital integration (HL7/FHIR feeds), credentialed full MIMIC-IV, clinical validation, real file uploads to the vault, and multi-instance deployment. **Intended users:** students, teachers and evaluators; the physician/patient/administrator roles are demonstration roles. **Operating environment:** Python 3.12 on Windows or Linux, a modern browser, SQLite. **Expected scale:** a single server process serving tens of concurrent users (Chapter 8).

1.6 Proposed Solution

A layered Flask application (Chapter 4) serves Jinja2/JavaScript portals and a JSON API. Synthetic predictions come from an in-memory ensemble whose weighted soft votes are fused into one of four tiers; a decision-tree rule path and k nearest historical cases make each result inspectable. The vault issues HS256 JSON Web Tokens [10] (HMAC [11]) whose server-side record holds the authoritative scope, expiry and use count. A separate research pipeline builds a hashed, versioned dataset from MIMIC-IV Demo files outside the repository, trains five pipelines under grouped cross-validation, and the web application serves only artifacts whose SHA-256 hashes verify.

1.7 Novelty and Originality

The individual algorithms are standard; the contribution is the *integration and the engineering discipline*: (i) one prototype combines explainable triage, patient-controlled sharing and a research pipeline; (ii) research mode is fail-closed—it refuses unsafe secrets, non-loopback binding and tampered artifacts, never falls back to synthetic data and never calls an external AI service; (iii) the stay-level research view scores every ICU stay with an out-of-fold model and refuses to serve unless the recomputed predictions reproduce the verified run exactly; and (iv) timestamp anomalies in the outcome data are kept, flagged and analysed rather than silently dropped.

1.8 Feasibility Analysis

Technical: all components use mature open-source libraries (Flask, scikit-learn [12], pandas, DuckDB) and run on a laptop. **Economic:** no licence costs; the MIMIC-IV Demo is open access; Gemini use is optional. **Operational:** one command starts the demo (`run_demo.py`); the production entry point seeds and trains on first start. **Time and resources:** the repository history shows development between 6 and 26 October 2026 (Section 7.8).

1.9 Social Relevance and Practical Applications

MediHaven is a teaching and demonstration artefact: it illustrates early-warning dashboards, consent-based record sharing and responsible evaluation of clinical ML. Its practical value lies in education and in prototyping user interfaces and data-protection patterns, not in patient care.

1.10 Success Criteria and Key Performance Indicators

Table 1.1 lists the KPIs fixed for this project; Chapter 9 compares them with the measured values.

1.11 Organisation of the Report

Chapter 2 reviews related systems and literature; Chapter 3 specifies requirements; Chapter 4 presents the architecture and design; Chapter 5 covers algorithms and complexity; Chapter 6 maps the work to the curriculum; Chapter 7 describes the implementation; Chapters 8–10 report performance, testing and results; and Chapter 11 concludes.

Table 1.1: Key performance indicators and targets

KPI	Definition	Target	Measurement method
Functional correctness	passed / total automated tests	100 %	`pytest` on the full suite
Response time	median latency of a risk prediction, 1 client	$\leq$ 150 ms	timed HTTP requests to Waitress
Reliability	failed requests under 16 concurrent clients	0	3,200-request load test
Throughput	simple API requests per second, 16 clients	$\geq$ 100 req/s	load test, `/api/vitals`
Resource use	server working set under load	$\leq$ 512 MB	process-tree memory sampling
Synthetic model quality	hold-out accuracy on the synthetic split	$\geq$ 0.91	project benchmark report
Research evaluation	discrimination with honest uncertainty	report AUROC with 95% CI	grouped CV + bootstrap

2 Literature Review and Study of Existing Systems

2.1 Study of Existing Systems

Bedside scores. NEWS aggregates vital-sign thresholds into an early-warning score [1]; SAPS II estimates ICU mortality from 17 variables [2]. Both are transparent and cheap but coarse. **Proprietary EHR-embedded models** such as the Epic Sepsis Model are widely deployed, yet external validation can reveal much weaker performance than expected [8]. **Research benchmarks** built on MIMIC [4, 13] standardise tasks but are not applications. **Patient-controlled sharing** prototypes such as MedRec use a blockchain for permissions [9]. **LLM-based documentation** tools summarise clinical text [14], with recognised factuality and safety risks [15].

2.2 Review of Research Papers and Technical Resources

Eighteen peer-reviewed papers were reviewed; bibliographic details were verified against DOI, Crossref, PubMed, PMLR or JMLR records. Findings are quoted only from the published abstracts.

[P1] MIMIC-IV [6]. *Problem:* hospital data are archived for storage, not research. *Approach:* a de-identified EHR database from Beth Israel Deaconess Medical Center. *Limitation:* single centre. *Relation:* source of the demo subset used here [5], distributed via PhysioNet [16].

[P2] Harutyunyan et al. [13]. Four MIMIC-III benchmarks (mortality, length of stay, physiologic decline, phenotyping) with linear and neural baselines and multitask training. *Limitation:* one database. *Relation:* motivates MediHaven's fixed, versioned task definition.

[P3] Purushotham et al. [4]. Deep models compared with a Super Learner and SAPS II/SOFA on MIMIC-III; deep models performed best, especially on raw time series. *Relation:* explains why a small tabular baseline set is a deliberate, modest starting point.

[P4] Rajkomar et al. [3]. Deep models on whole EHRs in FHIR format (216,221 inpatients, two centres) reported AUROC 0.93–0.94 for in-hospital mortality. *Limitation:* large, hard-to-interpret models and two centres. *Relation:* an upper reference point; MediHaven's demo cohort is three orders of magnitude smaller.

[P5] Johnson, Pollard and Mark [7]. Reproducing 28 MIMIC mortality studies found that half differed in sample size by more than 25 %. *Relation:* MediHaven publishes exact cohort, label and exclusion rules with counts for every step.

[P6] Pirracchio et al. (SICULA) [17]. A Super Learner on MIMIC-II (24,508 patients) achieved cross-validated AUROC 0.85–0.88 versus 0.78 for SAPS II. *Limitation:* external validation on only 200 patients. *Relation:* ensemble learning and cross-validation, applied here at teaching scale.

[P7] SAPS II [2]. 17 variables, 13,152 patients from 137 ICUs; AUROC 0.88 (development) and 0.86 (validation). *Relation:* conceptual ancestor of the 24-hour physiological summaries used as features.

[P8] NEWS [1]. NEWS discriminated the combined 24-hour outcome with AUROC 0.873 against 0.736–0.834 for 33 other scores. *Limitation:* retrospective, one hospital. *Relation:* the synthetic dashboard's tiered early-warning presentation.

[P9] Wong et al. [8]. External validation of the Epic Sepsis Model on 38,455 hospitalisations: AUROC 0.63; it missed 67 % of sepsis cases while alerting on 18 % of patients. *Relation:* the reason MediHaven reports uncertainty and refuses clinical claims.

[P10] TRIPOD [18]. A 22-item reporting checklist for prediction models. *Relation:* guided what the research mode reports (population, outcome, predictors, validation).

[P11] Kaufman et al. [19]. Formal definition of leakage and "learn–predict separation". *Relation:* the forbidden-feature guard, the 24-hour window with storetime checks and fold-internal preprocessing.

[P12] Saito and Rehmsmeier [20]. On imbalanced data, precision–recall plots reflect performance better than ROC plots. *Relation:* average precision is reported beside AUROC (prevalence 10.3 %).

[P13] Van Calster et al. [21]. Poor calibration misleads clinical decisions; calibration curves need large samples. *Relation:* Brier scores are reported, and calibration is left as future work given 11 deaths.

[P14] Rudin [22]. Argues for inherently interpretable models in high-stakes settings (position paper). *Relation:* decision-tree rule paths and logistic coefficients are shown.

[P15] Sendak et al. [23]. Proposes a one-page "Model Facts" label telling clinicians how and when (not) to use a model. *Relation:* the research page's model card and disclaimers.

[P16] Azaria et al. (MedRec) [9]. Blockchain-based record access and permission management leaving data in providers' storage. *Limitation:* prototype, not field tested. *Relation:* MediHaven pursues patient-controlled access with simpler signed, expiring tokens.

[P17] Van Veen et al. [14]. Adapted LLMs on four summarisation tasks; physicians rated the best summaries equivalent (45 %) or superior (36 %) to experts, with an analysis of fabricated content. *Relation:* motivates optional, de-identified narrative synthesis.

[P18] Singhal et al. [15]. MultiMedQA and human evaluation; Flan-PaLM reached 67.6 % on MedQA but showed key gaps, and Med-PaLM remained inferior to clinicians. *Relation:* narratives are labelled as non-advice and never receive identifiers.

2.3 Comparative Analysis

Table 2.1: Comparison of existing approaches with MediHaven

Criterion	Bedside scores [1, 2]	Proprietary EHR models [8]	Research benchmarks [13]	MediHaven
Explainability	high (thresholds)	low	varies	rule path, votes, coefficients
Reproducibility	high	low (closed)	high	versioned, hashed artifacts
Uncertainty reported	rarely	rarely	sometimes	bootstrap 95% CI
Patient-controlled sharing	no	no	no	QR passes with audit log
Clinical validation	yes	yes	n/a	**none** (prototype)
Cost	free	licensed	free	free / open source

2.4 Limitations of Existing Systems

Threshold scores are coarse; black-box models are hard to audit and may underperform on new sites [8]; published cohorts are hard to reproduce [7]; leakage can inflate results [19]; record sharing is rarely patient-controlled; and LLM outputs can fabricate content [14].

2.5 Identified Engineering Gap

For teaching, no single open example combines explainable triage, consent-based sharing and an honest, leakage-aware evaluation on real de-identified ICU data with fail-closed serving. This is an engineering and educational gap, not a claim of a new clinical method.

2.6 Proposed Improvements

MediHaven addresses the gap with an integrated prototype whose research results are reproducible, uncertainty-aware and explicitly limited; Chapter 10 compares it with these systems.

3 Software Requirements Specification (SRS)

3.1 Purpose and Scope of the SRS

This SRS specifies the behaviour implemented in the MediHaven repository (branch `param`, October 2026). Requirements marked *future* are not implemented.

3.2 Intended Users and Stakeholders

Physicians (triage, charts, QR scanning), patients (vault, sharing, access log), administrators (census, reassignment, escalation, model telemetry), researchers (opt-in research mode on their own machine), and evaluators/teachers. Demonstration accounts exist for each role; production deployments must replace them (Section 4.7).

3.3 Overall Product Description

A Python/Flask server with a SQLite database serves browser portals and a JSON API. Synthetic mode is the default. Research mode is enabled only by `DATA_MODE=mimic`, runs on loopback only, and reads verified artifacts stored outside the repository. Optional dependency: the Google Gemini API (synthetic mode only).

3.4 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement (input → output)	Processing	Acceptance criterion
FR-01	Sign in with username/password → session and role dashboard	hash check, rate limit	wrong password 401; 11th attempt in 5 min returns 429
FR-02	Role-based access to portals and APIs	session role vs route	patient cannot read another patient (403)
FR-03	Triage roster with tiers and filters	DB query, latest prediction	100 synthetic patients listed with tiers
FR-04	Patient chart: tier, score, 4 model votes, vitals, rule path, similar cases	ensemble inference	every patient returns 4 votes; 72-h chart non-empty

ID	Requirement	Processing	Acceptance criterion
FR-05	Alert feed, acknowledgement, escalation dispatch	in-memory alert state	identity taken from the signed-in user
FR-06	Vault records: add and list by category	category whitelist	invalid category 422
FR-07	Share QR pass with scope, lifetime, uses	HS256 JWT + DB record	lifetime $\leq$ 30 days, uses $\leq$ 1000
FR-08	Redeem pass by token or image	signature, expiry, revocation, atomic quota	single-use pass granted once under 4 concurrent scans
FR-09	Revoke pass; view access log	ownership check	revoked pass is denied and logged
FR-10	Clinical narrative (handover/discharge)	de-identified Gemini prompt or local fallback	no identifier in the prompt
FR-11	Research: model card, dashboard, what-if estimate	verified artifacts only	forbidden/invalid inputs rejected (422)
FR-12	Research triage: all ICU stays with out-of-fold scores	recompute and verify OOF	mismatch with saved metrics gives 503
FR-13	Health endpoint	model and mode status	`GET /api/health` returns 200

3.5 Non-Functional Requirements

Table 3.2: Non-functional requirements

Category	Requirement (implemented)
Performance	median prediction latency $\leq$ 150 ms for one client; simple reads $\geq$ 100 req/s
Scalability	one process, thread pool of 8; documented single-instance limit (Chapter 8)
Reliability	fail-closed research serving; first-start bootstrap reseeds an empty or interrupted database
Security/privacy	strong secrets enforced off-loopback; hashed passwords; CSP and security headers; 8-hour sessions; no MIMIC identifiers in responses or logs; research mode never calls external AI
Usability	consistent dark clinical theme; keyboard Esc closes dialogs; responsive header below 900 px
Maintainability	modular blueprints and services; 354 automated tests; pinned dependencies
Portability	Windows and Linux; Docker image recipe (Python 3.12)
Resource use	server working set $\leq$ 512 MB; database about 1.2 MB, model files about 0.2 MB

3.6 Hardware and Software Requirements

Hardware (measured on): Intel Core i5-8250U (4 cores, 8 threads), 7.9 GB RAM, Windows 11. Minimum: 2 cores, 2 GB RAM, 1 GB disk. **Software:** Python 3.12; Flask 3.1, Werkzeug 3.1, Waitress 3.0.2; pandas 3.0, NumPy 2.5, SciPy 1.18, scikit-learn 1.9.1, joblib 1.6; DuckDB 1.5 and PyArrow 26 (research); PyJWT 2.15, qrcode 8.2, Pillow 12.3, OpenCV (headless) 5.0, pyzbar 0.1.9 (optional); pytest 9.1; Chart.js 4.4.1 (self-hosted). All versions are pinned in `requirements.txt`.

3.7 Constraints and Assumptions

- **Data:** only synthetic patients in the web demo; MIMIC-IV Demo data stay outside the repository and OneDrive and are never sent to external services (data use terms).
- **Technical:** SQLite and in-process state imply a single server instance; research artifacts are tied to the exact scikit-learn version used for training.
- **Time/cost:** one academic semester; free and open-source tools only.
- **Assumption:** evaluators run the system locally or on one small server with HTTPS.

3.8 Use Cases and User Stories

Figure 3.1 summarises the use cases by role. Representative user stories: *As a physician*, I

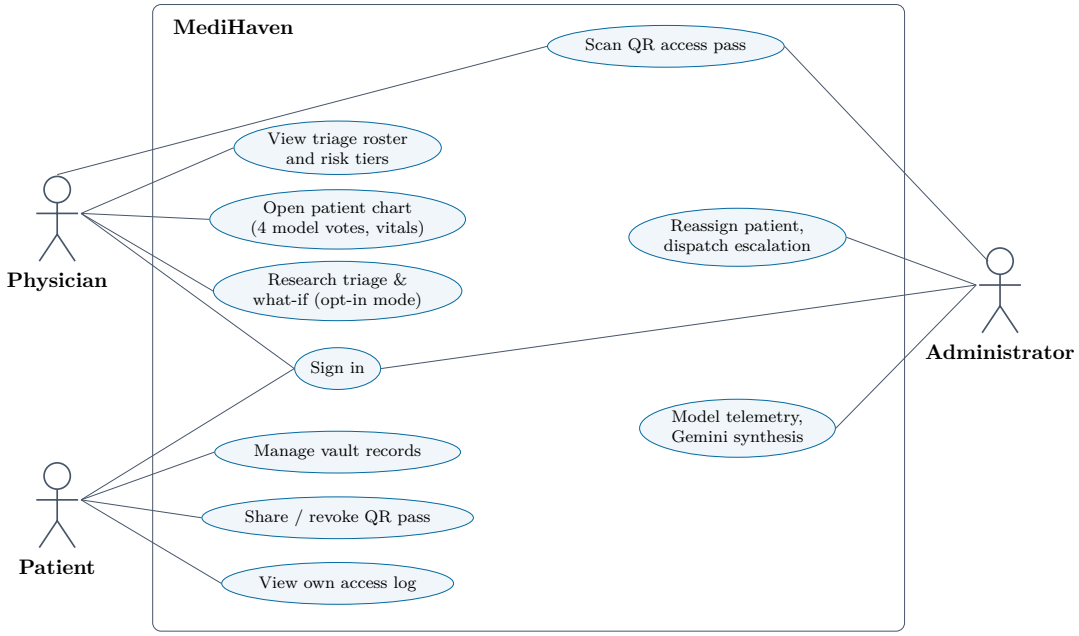


Figure 3.1: Use-case diagram

open a critical patient's chart and see why each model voted as it did. *As a patient*, I share

only my allergies and medications for one hour with a single-use pass and later see who used it. *As a researcher*, I see each ICU stay's out-of-fold risk next to its actual outcome without any MIMIC identifier leaving my computer.

4 System Architecture and Design

4.1 Overall System Architecture

MediHaven follows a layered client–server architecture (Figure 4.1). The browser renders server-side Jinja2 templates and calls the JSON API with `fetch`; the Flask application factory (`src/api/app.py`) wires security middleware and six blueprints; services hold the ML models, the vault logic, the narrative synthesiser and the research service; SQLite, model files and the research artifacts form the data layer.

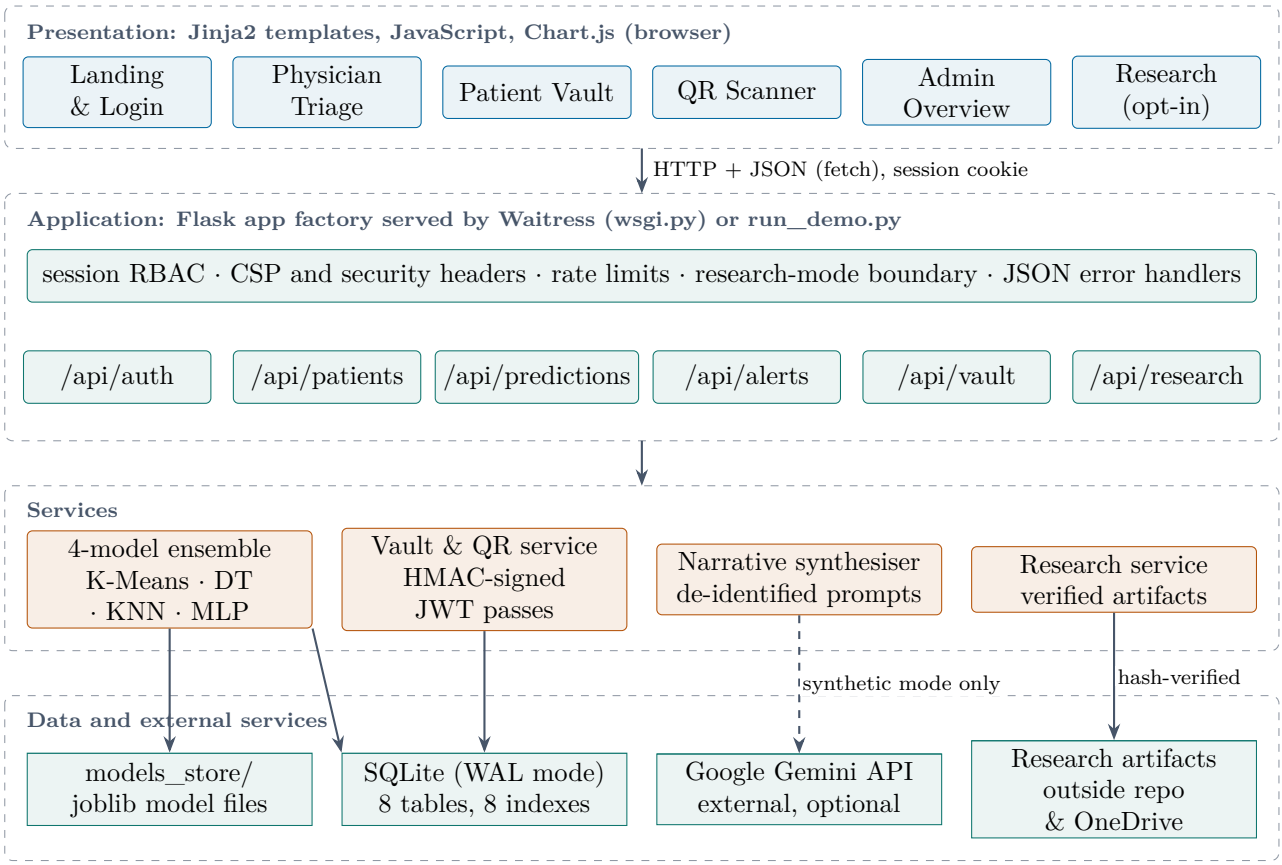


Figure 4.1: Layered architecture of MediHaven

Table 4.1: Major modules: purpose, inputs, outputs and dependencies

Module	Purpose	Input → output	Depends on
`api/app.py`, `auth.py`	app factory, RBAC, headers, rate limits	HTTP request → routed response	config, security
`routes/patients, predictions, alerts`	triage and charts	patient id → JSON packet	ensemble, SQLite
`models/ensemble.py` + 4 models	synthetic risk inference	30 features → tier, score, votes	scikit-learn
`vault/*`, `routes/vault`	records, QR passes, audit	scope, token → scoped records	PyJWT, OpenCV
`nlp/summarizer.py`	narratives	coarse model outputs → text	Gemini API (optional)
`tasks/build_mimic_t1.py`	research dataset	MIMIC CSV → hashed parquet	DuckDB, pandas
`training/*`	research training, verification, serving	dataset → verified run, OOF view	scikit-learn
`bootstrap.py`, `wsgi.py`	first start, production entry	config → ready app	Waitress

4.2 Component and Module Description

4.3 Architecture Selection and Justification

A layered, modular monolith was chosen over microservices: the workload is small (Chapter 8), the team is three students, models load once into process memory (no network hop per prediction), and a single SQLite file keeps deployment trivial. Blueprints and service classes keep the layers separable, so the data layer or the model service could later be extracted.

4.4 Data Flow Diagrams

Figure 4.2 shows the context and Figure 4.3 the level-1 processes and stores. The Gemini flow is dashed because it is optional and only ever receives coarse model outputs.

4.5 UML Diagrams

The use-case diagram appears in Figure 3.1. Figure 4.4 shows the main classes: the ensemble aggregates four model wrappers, and the research service composes the verified artifacts and the lazily built stay view. Figure 4.5 traces the redemption of a QR pass; the use count is claimed with a single conditional `UPDATE`, so concurrent scans cannot overspend a pass. Figure 4.6

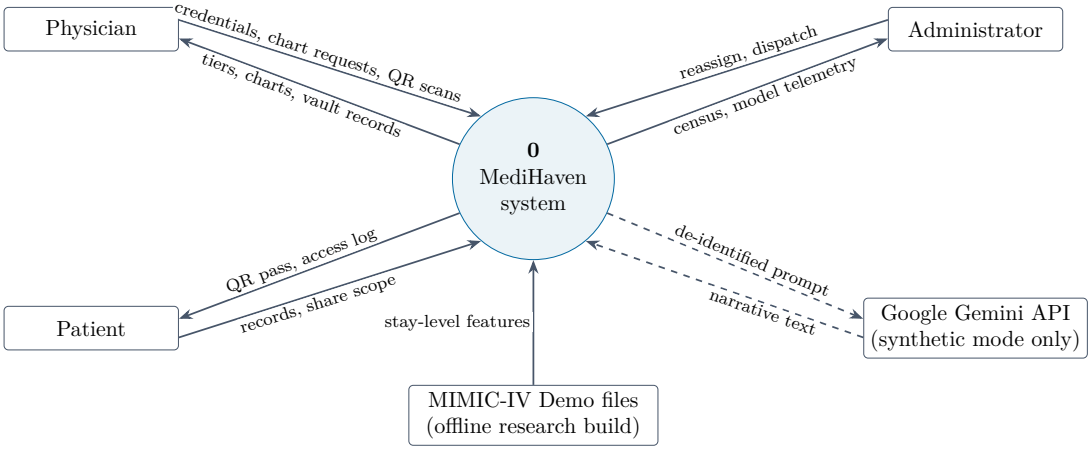


Figure 4.2: Context-level (level-0) data flow diagram

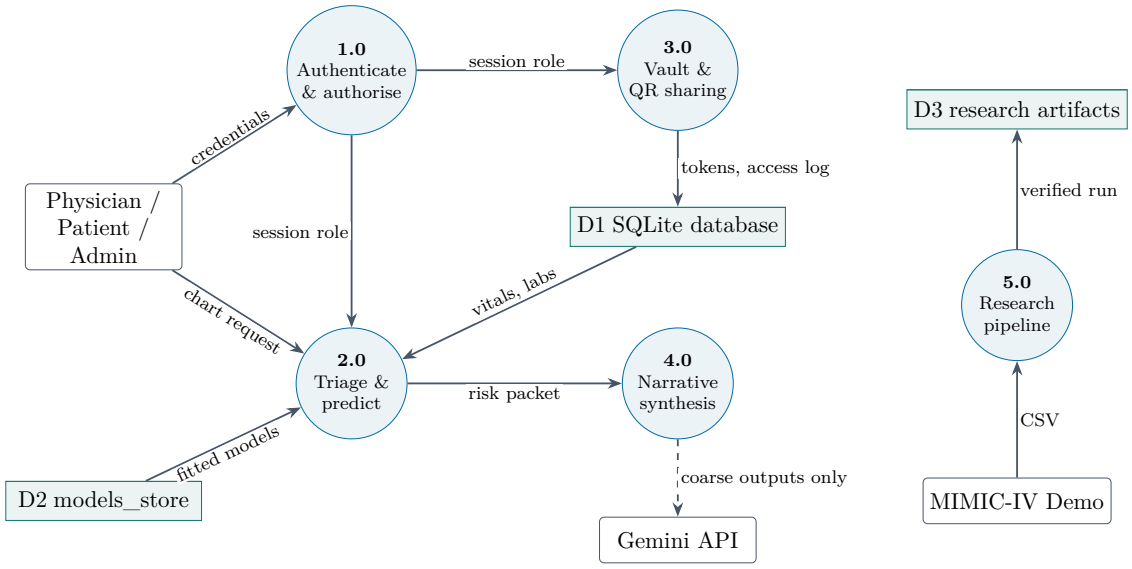


Figure 4.3: Level-1 data flow diagram

gives the activity flow of a prediction, Figure 4.7 the life cycle of a pass, and Figure 4.8 the deployment options.

4.6 Database Design

The schema (`database/schema.sql`, Figure 4.9) has eight tables in two domains: clinical (patients, vitals, lab results, medications, predictions, outcomes) and vault (records, access tokens, access log). Every table has a surrogate integer primary key; every child table references `patients(patient_id)` with `ON DELETE CASCADE`, and the access log also references the token. Each non-key attribute depends only on its table's key, so the schema is in third normal form; JSON columns (`scope_json`, `structured_data`) hold small, opaque lists that are never queried by element.

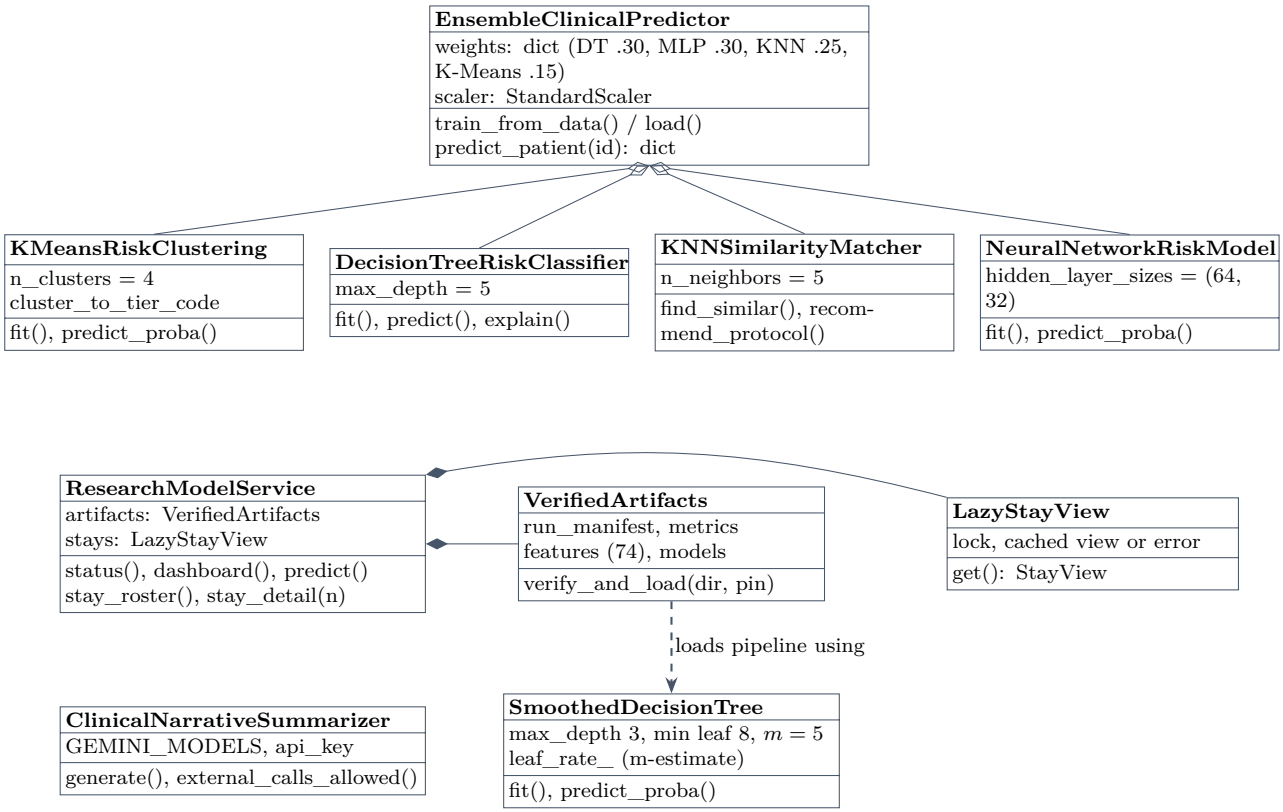


Figure 4.4: Class diagram of the model and service layer

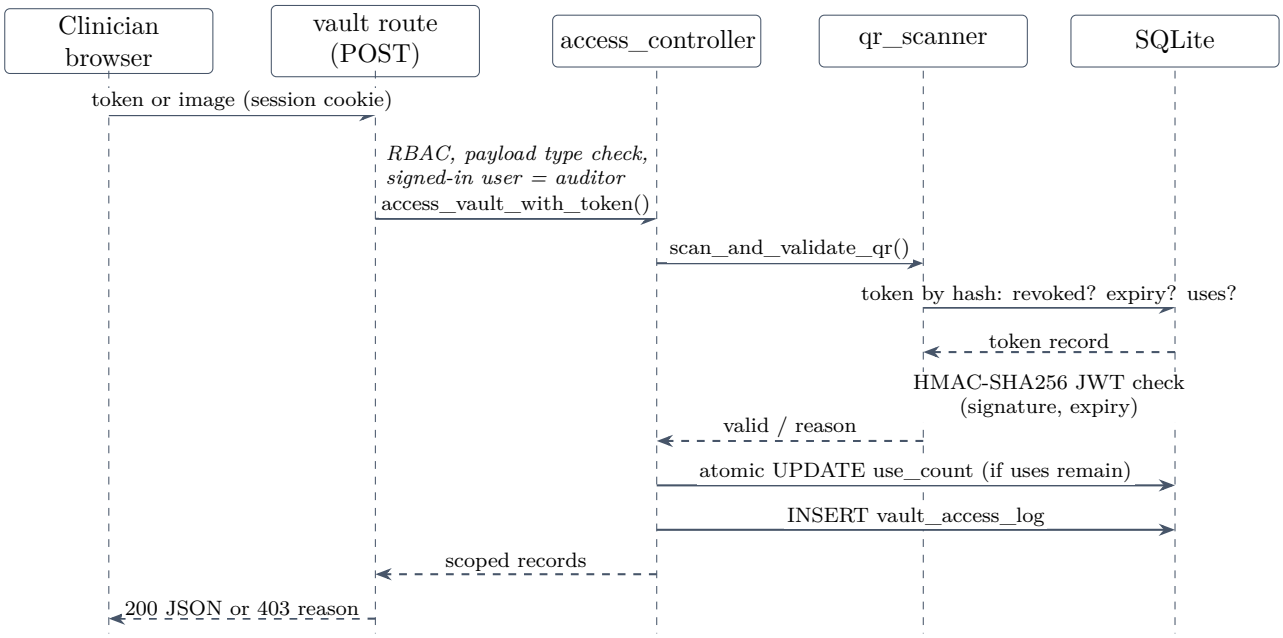


Figure 4.5: Sequence diagram: redeeming a QR access pass

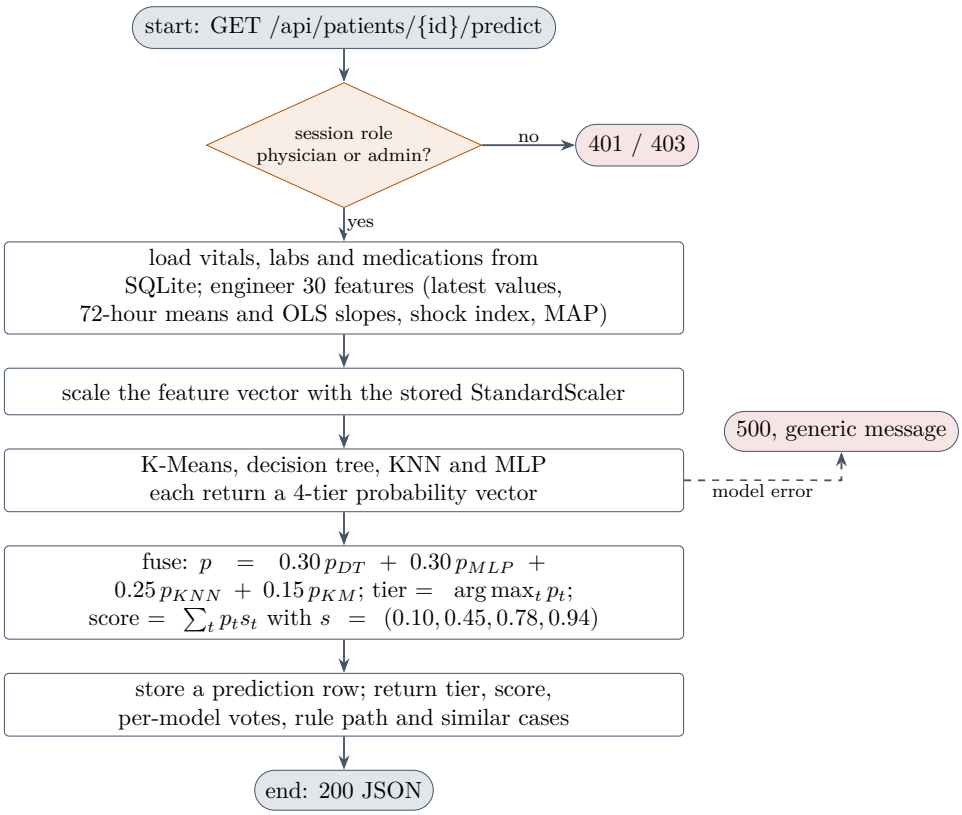


Figure 4.6: Activity diagram: synthetic risk prediction

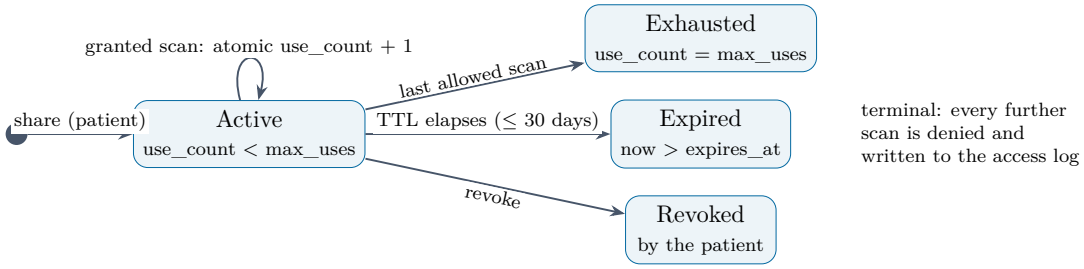


Figure 4.7: State machine of a QR access pass

Integrity constraints. `CHECK` constraints bound physiology (e.g. heart rate 20–260 bpm, SpO_2 50–100 %), enumerations (gender, status, route, vault category, access status) and probabilities ($0 \leq$ risk score ≤ 1); `mrn` and `token_hash` are `UNIQUE`. **Indexing.** Eight composite indexes follow the access paths: time series by `(patient_id, recorded_at DESC)` for vitals and labs, census by `(status, ward)`, vault by `(patient_id, category)`, token lookup by `(token_hash, revoked, expires_at)`, and the access log by patient and by token with `accessed_at DESC`. **Representative data.** On 26 October 2026 the local demo database held 100 patients, 4,368 vitals rows, 200 lab rows, 255 medications, 338 vault records and 210 issued passes; for example patient 1 (MRN-2026-0001, 53 years, M, Surgical Recovery, Bed-301) had a latest reading of HR 65.1 bpm, BP 121/80 mmHg, SpO_2 98.5 %. All of these are synthetic.

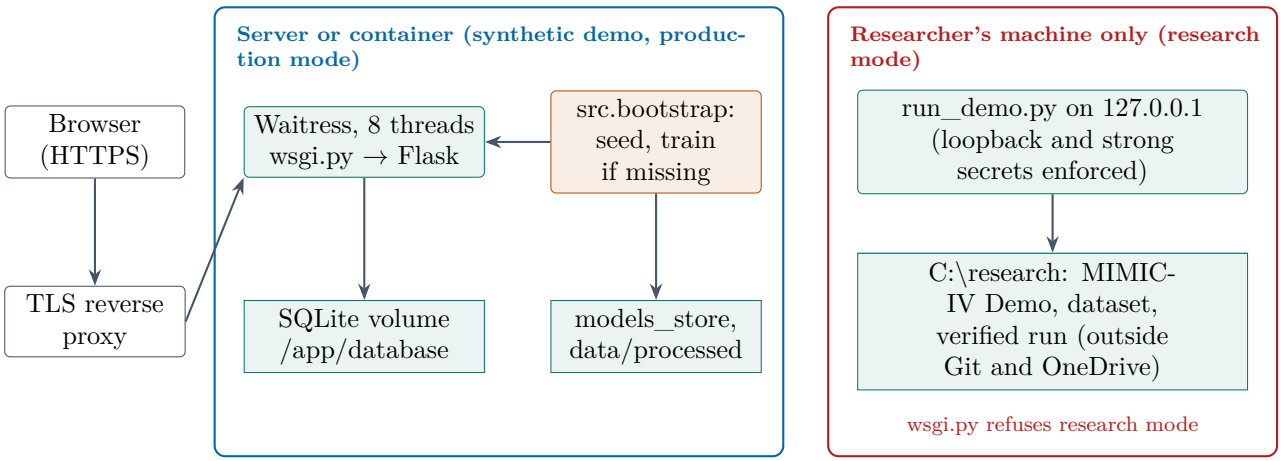


Figure 4.8: Deployment diagram: production (synthetic) and local research mode

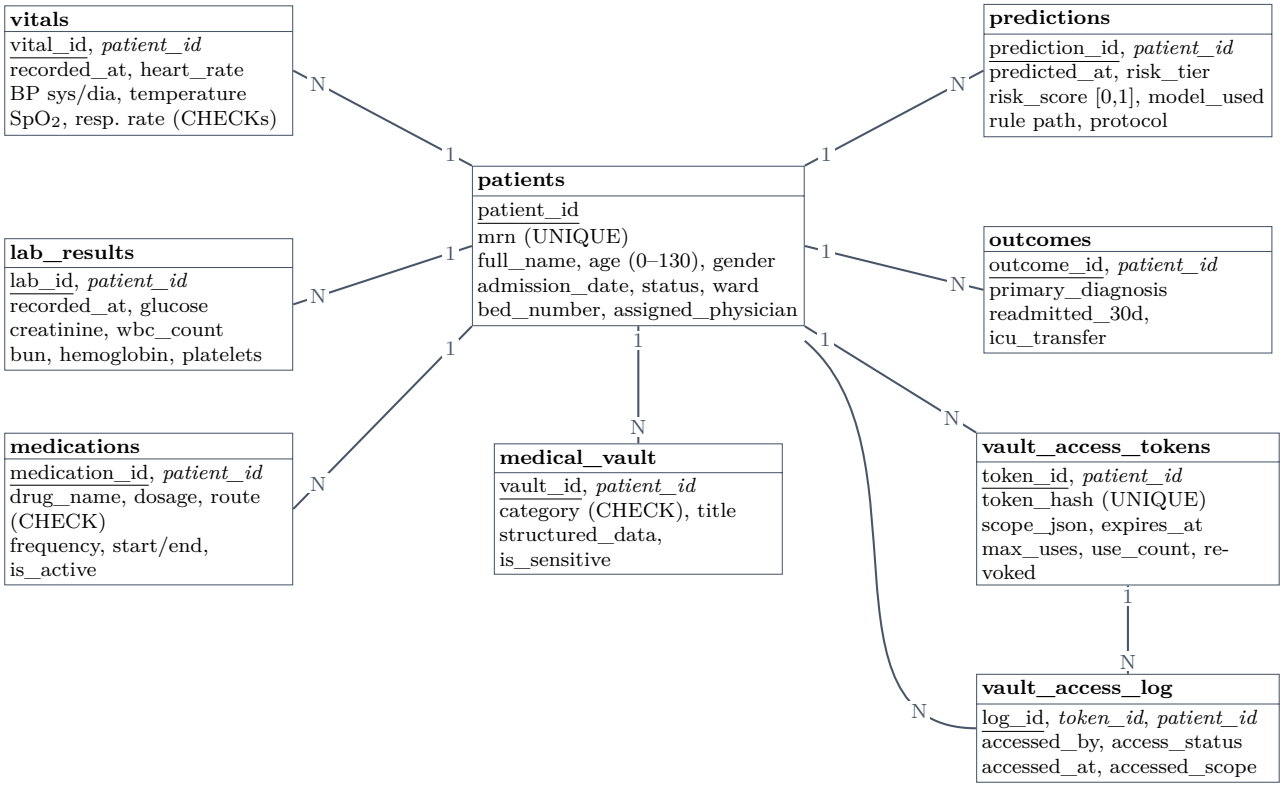


Figure 4.9: Entity–relationship diagram (PK underlined, FK in italics; 1:N relationships)

4.7 Security and Error Handling Design

- **Authentication:** only password hashes (scrypt, Werkzeug) are stored; in production, admin accounts need a hash supplied through the environment and the public demo logins need `DEMO_LOGINS=1`; login is limited to 10 attempts per 5 minutes per client.
- **Sessions:** signed cookies, HttpOnly, SameSite=Lax, Secure in production, 8-hour lifetime;

the application refuses default, placeholder or short secrets in production, on any non-loopback host and in research mode.

- **Authorisation:** role checks on every page and API; patients can reach only their own vault; audit entries take the clinician's identity from the session, never from the request body.
- **Input validation:** JSON object bodies only; bounded integers (pass lifetime $\leq$ 30 days, uses $\leq$ 1000); scanner payloads must be a token or an image data URI no larger than 4096×4096 pixels; requests above 8 MB are rejected (413).
- **Browser hardening:** Content-Security-Policy allowing only same-origin scripts (Chart.js is self-hosted, no inline scripts), `X-Frame-Options: DENY`, `nosniff`, HSTS over HTTPS, `no-store` on API responses.
- **Errors and logging:** JSON error envelopes with generic messages; details are logged server-side only; logging falls back to the console when the log folder is not writable.
- **Sensitive data:** Gemini receives only coarse outputs with a `[PATIENT]` placeholder over verified TLS; research mode never calls it, refuses non-loopback binding, verifies SHA-256 hashes before deserialising any model, and never returns MIMIC identifiers or dates.

5 Algorithm Design and Data Structures

5.1 Identification of Critical Operations

(1) feature engineering for a synthetic patient; (2) ensemble inference and fusion; (3) QR pass validation and atomic redemption; (4) research cohort, labelling and leakage-safe feature extraction; (5) patient-grouped cross-validation with bootstrap intervals; (6) verified serving of research artifacts and out-of-fold stay scores; (7) request rate limiting.

5.2 Algorithm Design

Feature engineering. For each synthetic patient, 30 features are computed: demographics, latest vitals, 72-hour means and minima (window ending at the latest reading), compound indices (mean arterial pressure MAP = DBP + (SBP − DBP)/3, pulse pressure, shock index HR/SBP), laboratory values and deltas, and the active-medication count. Trends use the ordinary-least-squares slope

$$\beta_1 = \frac{\sum_i (t_i - \bar{t})(y_i - \bar{y})}{\sum_i (t_i - \bar{t})^2} \quad \text{(units per hour, 0 if fewer than two points).} \tag{5.1}$$

Synthetic models (scikit-learn [12], fixed seed 42):

- *K-Means* ($k = 4$, k-means++, 10 restarts, ≤300 iterations): clusters are mapped to tiers by centroid severity; distances d_j become pseudo-probabilities $\propto 1/(1 + d_j)$, re-ordered by tier.
- *Decision tree* (CART, Gini, depth ≤ 5, ≥ 2 samples per leaf) on unscaled values, so its root-to-leaf rule path is readable in clinical units.
- *KNN* ($k = 5$, Euclidean distance on scaled features, scikit-learn `algorithm='auto'`): similarity-weighted votes of the nearest synthetic cases, which also supply the protocol suggestion.
- *MLP* (hidden layers 64 and 32, ReLU, Adam, $\alpha = 10^{-3}$, batch 16, early stopping on 15 % validation data, ≤ 500 epochs).

Algorithm 1 Ensemble fusion (`EnsembleClinicalPredictor.predict`)

Require: scaled feature vector x, raw vector x_{raw}

1: $p_{KM} \leftarrow$ tier-ordered $1/(1 + d_j)$ of x to the 4 centroids, normalised
2: $p_{DT} \leftarrow \textsc{TreeProba}(x_{raw})$; $p_{KNN} \leftarrow$ similarity-weighted votes of 5 neighbours
3: $p_{MLP} \leftarrow \textsc{MLPProba}(x)$
4: $p \leftarrow 0.30\, p_{DT} + 0.30\, p_{MLP} + 0.25\, p_{KNN} + 0.15\, p_{KM}$
5: tier $\leftarrow \arg\max_t p_t$; score $\leftarrow \sum_t p_t s_t$ with $s = (0.10, 0.45, 0.78, 0.94)$
6: **return** tier, score, confidence $= \max_t p_t$, and each model's own tier and score

Algorithm 2 QR pass redemption (`access_vault_with_token`)

Require: token string or image, signed-in clinician u

1: decode the QR image if needed (size $\leq 4096^2$ pixels); verify HS256 signature and expiry
2: look up the record by token hash; deny if revoked, expired or out of uses
3: **atomic** `UPDATE tokens SET use_count=use_count+1 WHERE id=? AND revoked=0 AND (max_uses IS NULL OR use_count<max_uses)`
4: **if** no row changed **then** log MAX_USES_EXCEEDED and deny
5: **end if**
6: read the scope from the stored record; return only matching vault records; append u to the access log

Research pipeline. Figure 5.1 shows the cohort and label construction; features use only observations with `charttime` and `storetime` at or before the index time (intime + 24 h), which removes look-ahead leakage [19]. Model set v2.0.0 contains a prior-only reference and four models, each a pipeline of median imputation, removal of constant columns and (where needed) standardisation: L2 logistic regression ($C = 0.1$); a *smoothed* decision tree (depth 3, ≥ 8 stays per leaf) whose leaf risk is the m-estimate

$$\hat{p}_{\text{leaf}} = \frac{\text{deaths}_{\text{leaf}} + m\,\hat{p}}{n_{\text{leaf}} + m}, \qquad m = 5, \tag{5.2}$$

so that no leaf predicts exactly 0 or 1; a random forest (300 trees, depth ≤ 4); and an MLP with one hidden layer of 16 units ($\alpha = 1$, L-BFGS). The 14 measurement-count features are excluded from model inputs (74 inputs remain) because they encode monitoring intensity rather than physiology.

Out-of-fold stay scores. For the research triage view, the five-fold split is recomputed with the recorded seed, each stay is scored by the pipelines trained on the other folds, and the view is served only if the fold-assignment fingerprint and every model's pooled AUROC equal the verified run (tolerance 10^{-9}); otherwise it fails closed.

5.3 Flowcharts

Flowcharts for the prediction activity (Figure 4.6) and the research pipeline (Figure 5.1) cover the two most complex operations.

5.4 Data Structure Selection

- **Arrays (NumPy):** feature matrices and probability vectors give vectorised arithmetic.
- **Hash tables (Python `dict`/`set`):** credential store, tier maps, stay lookup by number, identifier sets for leak checks; $O(1)$ expected lookup.
- **Trees:** the decision tree itself; B-tree indexes in SQLite make range scans on `(patient_id, recorded_at)` logarithmic.

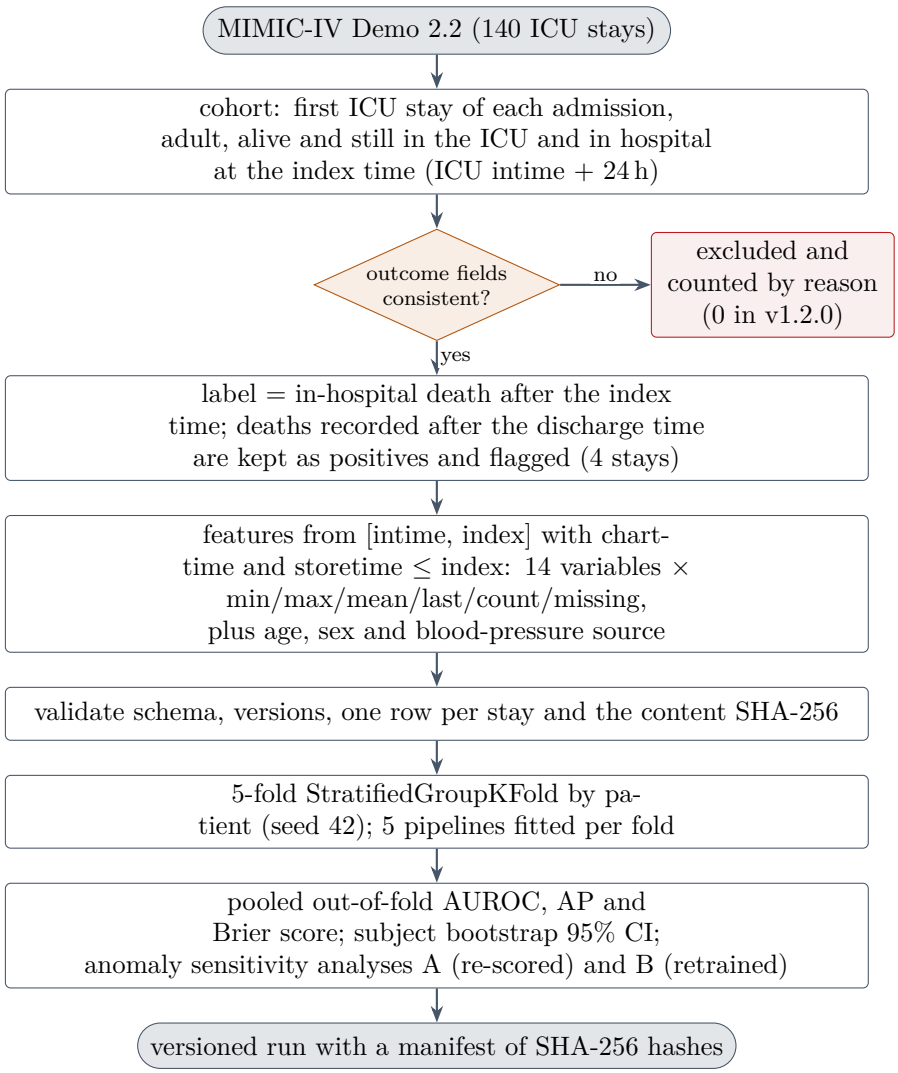


Figure 5.1: Flowchart of the research pipeline (dataset v1.2.0, task v3, run v2.0.0)

- **Queues:** the rate limiter keeps a `deque` of timestamps per client (sliding window, amortised $O(1)$ eviction); Waitress dispatches requests from a task queue to a pool of 8 threads.
- **Locks:** mutexes protect the rate-limit table and the lazily built stay view.

5.5 Algorithm Selection and Justification

Classical models were preferred to deep networks: the synthetic set has 100 patients and the research cohort 107 stays, far too few for deep models, and trees, coefficients and neighbours are explainable [22]. K-Means adds an unsupervised phenotype view; KNN supplies case-based precedents. Patient-grouped stratified folds were chosen over a hold-out split because a split of 107 stays would contain only one or two deaths, and grouping prevents one patient's stays appearing in both training and test folds.

5.6 Time Complexity Analysis

With n training samples, d features, k clusters/neighbours, T trees and E epochs:

Table 5.1: Theoretical complexity of the main algorithms

Algorithm	Training	Inference (one sample)
K-Means (Lloyd)	$O(n\,k\,d\,I)$ per restart	$O(k\,d)$
Decision tree (CART)	$O(d\,n \log n)$	$O(\text{depth}) \leq O(5)$
KNN (brute force)	$O(1)$ (store)	$O(n\,d)$, plus $O(n)$ selection
MLP	$O(E\,n\,(d \cdot 64 + 64 \cdot 32))$	$O(d \cdot 64 + 64 \cdot 32)$
Random forest	$O(T\,d'\,n \log n)$, $d' = \sqrt{d}$	$O(T \cdot \text{depth})$
Grouped 5-fold CV	$5\times$ training of all 5 pipelines	–
Subject bootstrap (B resamples)	$O(B\,n \log n)$ for AUROC	–
QR redemption	–	$O(\log N)$ index lookups + $O(r)$ records

These are theoretical bounds; measured run times are reported in Chapter 8.

5.7 Space Complexity Analysis

Model storage is $O(kd)$ (K-Means), $O(\text{nodes})$ (tree), $O(nd)$ (KNN keeps the training set) and $O(\text{weights})$ (MLP); all synthetic model files together occupy about 0.22 MB. The research stay view holds 107 rows of 14 variable summaries in memory, and the bootstrap stores B metric values per metric.

5.8 Optimisation Opportunities

Prediction requests recompute features from the database each time (Section 8.7); caching feature vectors per patient and invalidating on new vitals would remove most of that cost. KNN could use a pre-built KD-tree for larger cohorts, and inference could move to a separate worker process to escape Python's global interpreter lock.

6 Subject-Wise Engineering Concepts and APNI Leap Alignment

The Report Preparation Guidelines ask each group to show the APNI Leap emphasis—engineering fundamentals, practical implementation, software engineering practice, algorithmic efficiency, scalability, testing and measurable performance—and to map Semester I–IV subjects to real project evidence. The subject codes below are those recorded in the project documentation; this chapter does not claim any formal APNI Leap assessment or endorsement.

6.1 Software Engineering (CE202)

Modular blueprints and service classes; requirement identifiers traced to tests (Table 9.1); centralised exception handling with JSON error envelopes; configuration through environment variables; version control with feature branches and pull requests (Section 7.8); pinned dependencies and a reproducible build (Dockerfile).

6.2 Computer Networks

MediHaven is a client–server application over HTTP/1.1 on TCP. The browser exchanges JSON messages (`{success, data, error, meta}` envelopes) with the server; Waitress accepts TCP connections and serves concurrent clients with a pool of 8 threads; the session cookie carries authenticated state; TLS is terminated by a reverse proxy in production. Error recovery is at the application level (HTTP status codes, redirect to login on 401). No custom socket protocol was implemented.

6.3 Database Management Systems (CE204)

A normalised (3NF) relational schema with primary/foreign keys, cascading deletes, `CHECK` and `UNIQUE` constraints, and eight composite indexes matching the access paths (Section 4.6). SQLite runs in write-ahead-logging (WAL) mode so readers do not block the writer; every request opens its own connection; transactions commit atomically (the seeding script commits once, so a crash leaves no partial cohort). The QR quota uses a conditional `UPDATE` as an atomic compare-and-increment.

6.4 Operating Systems (CE205)

Thread-based concurrency (Waitress worker pool), mutual exclusion with locks (rate-limit table, lazily built stay view), the race condition found and fixed in pass redemption (Section 9.6), background threads for warm-up, and process-tree resource measurement (Chapter 8). Scheduling is delegated to the operating system and the WSGI thread pool; no custom scheduling policy (e.g. Round Robin) was implemented.

6.5 Data Structures (CE207)

Arrays for feature matrices, hash tables for constant-time lookups, trees (decision tree, B-tree indexes), a sliding-window queue per client for rate limiting, and a task queue in front of the thread pool (Section 5.4).

6.6 Design and Analysis of Algorithms (CE207)

CART, k-means (Lloyd's algorithm), nearest-neighbour search, back-propagation, stratified group k-fold splitting and the subject bootstrap, with correctness arguments and complexity bounds in Chapter 5.

6.7 Applied Mathematics and Statistics (AS202)

Least-squares slopes, derived haemodynamic indices, standardisation, probabilities and soft voting, AUROC, average precision, Brier score and bootstrap confidence intervals.

6.8 Cloud Computing

The project provides deployment-ready artefacts—a Waitress entry point, a non-root Docker image recipe with a health check, environment-based secrets and proxy support—but it has *not* been deployed to a cloud provider, so no cloud resource figures are reported.

6.9 Mapping of Subjects to Project Implementation

Table 6.1: Mapping of subjects to project modules and evidence

Subject	Concept applied	Project module	Evidence
DBMS	keys, constraints, composite indexes, atomic update	`database/schema.sql`, `vault/access_controller.py`	schema; concurrency test (4 scans, 1 grant)
Operating Systems	thread pool, locks, race-condition fix	`wsgi.py`, `api/rate_limit.py`	load test (Ch. 8); rate-limit tests
Computer Networks	HTTP client–server, JSON messages, sessions	`api/*`, `static/js/*`	44 routes; API tests
Data Structures	hash maps, deques, trees	rate limiter, stay view, models	code; complexity table
Algorithms	CART, k-means, KNN, grouped CV, bootstrap	`models/*`, `training/*`	pseudocode; metrics with CIs
Applied Mathematics	OLS slopes, AUROC, Brier, bootstrap	`data/feature_engineering.py`, `training/metrics.py`	unit tests; evaluation outputs
Software Engineering	modularity, testing, version control	whole repository	354 passing tests; 5 merged PRs

7 Implementation and Module-Wise Coding

7.1 Development Environment

Python 3.12 (virtual environment), VS Code, Git and GitHub, Windows 11; Flask 3.1 with Jinja2, vanilla JavaScript and Chart.js 4.4.1; scikit-learn 1.9.1, pandas 3.0, NumPy 2.5; SQLite 3 (WAL); DuckDB 1.5 for reading the MIMIC-IV CSV files; pytest 9.1. AI coding assistants were used during development: 13 of the 25 non-merge commits carry an AI co-author trailer, and all such changes were reviewed and tested before being committed.

7.2 Overall Implementation Strategy

The repository history shows the order of work: foundation and configuration, database schema and seeding, feature pipeline, the four-model ensemble, the vault and QR engine, the REST API, the user interface and a benchmark runner (6–7 October 2026); RBAC and the Gemini narrative (8 October); a security and RBAC fix (14 October); then the research track—test isolation and data modes, the MIMIC-IV adapter, the Task 1 dataset, leakage-aware baselines, verified research serving and security fixes (25 October)—followed by the research dashboard, model set v2, deployment hardening and a vitals-chart fix (26 October).

7.3 Module-Wise Implementation

Table 7.1: Module-wise implementation summary

Module	Processing logic and key details	Code references	Test evidence
Data pipeline	clean, validate, engineer 30 features, stratified 80/20 split, fit and save scaler	`src/data/*`	`test_data_pipeline.py`
Ensemble	train four models, fuse soft votes, rule path, KNN precedents, per-model scores	`src/models/*`	`test_models.py`, `test_benchmark.py`
API and RBAC	app factory, blueprints, session roles, error envelopes, rate limits, headers	`src/api/*`	`test_api.py`, `test_auth.py`
Vault and QR	HS256 JWT with DB record, scope filter, atomic quota, audit log	`src/vault/*`	`test_vault.py`
Narratives	de-identified prompt, TLS-verified call, local fallback	`src/nlp/summarizer.py`	`test_nlp.py`, `test_security_hardening.py`

Module	Processing logic and key details	Code references	Test evidence
MIMIC adapter and Task 1	DuckDB reads, item-ID mapping, unit checks, cohort, labels, anomalies, hashed dataset	`src/datasources/mimic/*`, `src/tasks/*`	`test_mimic_t1_dataset.py`, `test_mimic_layout_mappings.py`
Research training	grouped CV, metrics, bootstrap, sensitivity analyses, staged atomic output	`src/training/*`	`test_training_t1.py`
Research serving	hash-verified loading, what-if contract, dashboard, out-of-fold stay view	`training/serving.py`, `stays.py`, `whatif.py`	`test_research_mode_app.py`, `test_whatif_contract.py`
Deployment	Waitress entry, first-start bootstrap, Docker image	`wsgi.py`, `src/bootstrap.py`, `Dockerfile`	`test_deployment_hardening.py`

7.4 Front-End Implementation

Seven server-rendered portals share `base.html` (role-aware navigation, responsive header): landing, login, physician triage (KPI cards, alert feed, filterable roster, chart drawer with Chart.js vitals and model votes), patient vault (records, sharing dialog, QR image, access log), scanner (camera, upload or pasted token), administrator overview, and in research mode the research triage and model analytics pages. All scripts are static files: a shared `common.js` redirects to the login page on an expired session, research scripts build the DOM with `textContent` only, and no data are kept in browser storage. Screenshots appear in Chapter 10.

7.5 Back-End Implementation

The 44 routes (Appendix A) follow one pattern: authenticate and authorise, validate the JSON body (objects only, bounded values), call a service, and return a `{success, data, error, meta}` envelope. Models are loaded once into application memory at start-up; research mode replaces them with a `ResearchModelService` that verifies every artifact's SHA-256 hash before deserialisation and blocks the synthetic clinical APIs with HTTP 503.

7.6 Database Implementation

`src/database/db.py` opens one connection per operation with `PRAGMA foreign_keys=ON`, `journal_mode=WAL` and `synchronous=NORMAL`, and returns dictionary-like rows; all SQL is pa-

rameterised. Research mode opens the demo database read-only (`mode=ro`) and never creates it.

7.7 Communication and Concurrency Implementation

Clients communicate over HTTP with JSON; Waitress serves them with 8 worker threads. Shared state is limited and protected: rate-limit counters and the research stay view use locks; alert acknowledgements are kept per process, which is why a single instance is required. The redemption race (several concurrent scans of a single-use pass all succeeding) was removed with an atomic conditional update and is covered by a concurrency test.

7.8 Git Repository Setup

Repository: `https://github.com/param7862989-ops/MediH` (30 commits; 144 tracked files; about 24,400 lines of Python, JavaScript, HTML, CSS and SQL including tests). **Branching:** `main` is the integration branch; work is done on personal feature branches (`nirupam`, `param`) and merged through pull requests (five merged, #1–#5). **Commit convention:** Conventional Commits (`feat(scope):`, `fix(scope):`, `docs:`). **History by Git identity:** `paramthakkar24-sudo` 13 commits, `Nirupam` 10, `param7862989-ops` 7 (including the five merge commits). Git identities are not verified mappings to students; individual contributions are to be confirmed by the group (*see MISSING_INFORMATION.md*). At the time of writing, the research triage page (stay-level out-of-fold view) is implemented and tested in the working tree of branch `param` but not yet committed.

7.9 Integration of Modules

`create_app()` integrates configuration, security checks, middleware, model loading and the blueprints; `src/bootstrap.py` chains seeding → feature pipeline → ensemble training when assets are missing; `run_demo.py` (local) and `wsgi.py` (production) are the two entry points. The research pipeline is integrated only through its verified on-disk run.

7.10 Implementation Challenges and Resolutions

- **Look-ahead leakage:** a value can be charted inside the 24-hour window but stored in the record only later; the reader therefore requires `storetime` $\leq$ index time as well (568 such observations excluded on the demo).
- **Outcome timestamp anomalies:** four deaths are recorded after discharge; they are labelled positive, flagged and analysed separately instead of being silently dropped.
- **Misleading what-if inputs:** measurement counts reflected monitoring intensity and could not be entered meaningfully; model set v2 excluded them (74 inputs).

- **Concurrency defect:** single-use QR passes could be redeemed more than once; fixed with an atomic update.
- **Stale demo data:** the 72-hour vitals chart became empty once the seeded data were more than three days old; the window now ends at each patient's latest reading.

8 Scalability, Performance and Resource Management

8.1 Scalability Requirements

MediHaven targets classroom-scale use: one hospital-ward-sized synthetic cohort (100 patients, 4,368 vitals rows) and tens of concurrent users. The research cohort is 107 ICU stays. No requirement exists for multi-site or high-availability operation.

8.2 Workload and Load Model

Four representative endpoints were exercised: `/api/health` (no database), `/api/patients` (roster query), `/api/patients/{id}/vitals?hours=72` (indexed time-series read) and `/api/patients/{i` (feature engineering, four models, one insert). For each endpoint and each concurrency level $c \in \{1, 4, 8, 16\}$, 200 requests were spread over c client threads sharing one authenticated physician session, cycling over the 100 patients.

8.3 Performance Measurement Setup

Server: `wsgi.py` under Waitress 3.0.2 with 8 threads, `FLASK_ENV=production`, on a temporary copy of the synthetic database (the persistent database was not modified). Client: Python `urllib` on the same machine (loopback). Hardware: Intel Core i5-8250U (4 cores/8 threads), 7.9 GB RAM, Windows 11, Python 3.12. Latency is wall-clock time per request; memory and CPU time are sampled for the whole server process tree with PowerShell `Get-Process`. The script and raw results are included as `appendices/perf_measure.py` and `appendices/perf_results.json`. Two earlier runs showed the same overall pattern, but individual cells varied by up to a factor of two (for example vitals at 8 clients: 116 versus 226 req/s), so the figures indicate magnitudes, not precise capacities.

8.4 Performance Results

8.5 Resource Utilisation

The server process tree used 197.3 MB of memory when idle and at most 206.1 MB during the test, nearly all of it the Python runtime with pandas, NumPy and scikit-learn loaded. The synthetic database file is 1.17 MB and the model files 0.22 MB. Network traffic was loopback only and was not measured.

Table 8.1: Measured latency and throughput (200 requests per cell, 0 errors in all 3,200 requests)

Endpoint	Clients	Throughput (req/s)	p50 (ms)	p95 (ms)	Server CPU
health	1 / 16	77.4 / 670.7	15.2 / 21.8	25.3 / 32.5	0.25 / 0
patient list	1 / 4 / 16	43.8 / 93.9 / 35.3	21.8 / 37.6 / 447.6	36.8 / 73.0 / 505.5	1.9 / 3.8 / 1
vitals (72 h)	1 / 16	44.4 / 268.8	19.5 / 55.1	34.7 / 79.6	1.6 /
predict	1	13.5	71.9	99.6	7
predict	4 / 8 / 16	15.4 / 15.6 / 15.2	257.9 / 513.1 / 1056.9	310.1 / 599.8 / 1177.9	71.9 / 67.6 / 6

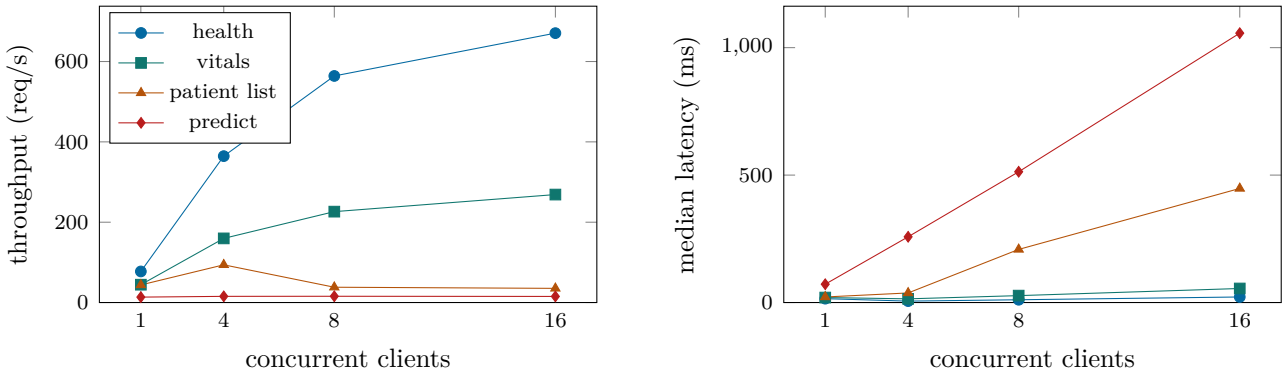


Figure 8.1: Throughput and median latency against concurrency (same legend in both panels)

8.6 Concurrent Processing, Scheduling and Synchronisation

Waitress queues incoming connections and dispatches them to 8 worker threads; the operating system schedules the threads. Python's global interpreter lock serialises pure-Python work, while NumPy and scikit-learn release it inside numerical kernels. Shared state is protected by locks, SQLite runs in WAL mode so reads proceed alongside a writer, and the pass quota is claimed with an atomic update. No custom scheduling policy is implemented.

8.7 Bottleneck Identification

Prediction is CPU-bound: 200 prediction requests consumed about 70 CPU-seconds (≈0.35 s each across several cores), so throughput stays near 15 requests/s at every concurrency level and latency grows linearly with the queue (72 ms at 1 client, about 1.06 s at 16). **The roster degrades under contention:** its throughput fell from 93.9 req/s (4 clients) to 35–38 req/s (8–16 clients) while CPU time per run tripled, which points to interpreter-lock contention between threads doing pure-Python row processing (an interpretation, not a profiled result). Simple reads (health, indexed vitals) scale well up to 16 clients.

8.8 Optimisation and Improvement

Already applied: models are loaded once and cached in memory; time-series reads use the composite index; static assets are self-hosted. Proposed (not implemented): cache per-patient feature vectors, move inference to worker processes, limit numerical-library threads per request, and paginate the roster.

8.9 Cloud Deployment and Resource Utilisation

A non-root Docker image recipe with a health check and environment-based secrets is provided, but no cloud deployment was carried out, so no cloud resource measurements exist.

8.10 Scalability Limitations

Maximum tested workload: 16 concurrent clients, 3,200 requests, all successful. A single instance is required because alert acknowledgements and rate-limit counters live in process memory and SQLite suits one host. Projected capacity beyond 16 clients is not established by these tests.

9 Testing and Validation

9.1 Testing Strategy

Testing is automated with pytest and organised in 19 modules (354 test cases including parametrised cases). A session fixture redirects the database, model store, processed data and logs to a temporary folder, pins public development secrets, and fails the run if any persistent file changes, so tests never touch real data or secrets. Research tests build their own small synthetic MIMIC-shaped fixtures and run the real training pipeline on them; no real MIMIC record is used in tests. The full suite was executed on 26 October 2026 on the working tree of branch `param`:

```
.venv\Scripts\python.exe -m pytest tests/ -q -p no:cacheprovider
```

Result: **354 passed, 0 failed, 0 skipped** in 313.75 s.

An earlier clean-state run recorded 301 passed; the increase reflects tests added since for deployment hardening and the research triage view.

9.2 Unit, Integration and Functional Testing

Unit tests cover each model (cluster alignment, tree rule paths, KNN precedents, MLP probabilities), feature engineering, metrics, splits, the smoothed tree and the what-if input contract. **Integration tests** drive the Flask app through its test client: API → services → SQLite, research serving over a real (fixture) Phase 2 run, and the vault from issuance to audit. **Functional tests** check each requirement in Table 3.1 (Table 9.1).

9.3 Non-Functional, Boundary and Invalid-Input Testing

Security tests verify TLS verification and de-identified prompts for Gemini, hashed passwords, refusal of weak secrets in production and off-loopback, security headers, absence of inline scripts (CSP), rate limits, cookie flags and CORS. Boundary and invalid-input tests cover JSON arrays instead of objects, `null` fields, huge numbers (pass lifetime 10^{12} minutes, `hours=999999999999`), boolean quotas, file paths sent to the scanner, oversized images, forbidden research features, contradictory what-if inputs, tampered artifacts and malformed manifests; all must return 4xx or fail closed, never 500.

9.4 Concurrency and Stress Testing

A concurrency test starts four threads that redeem the same single-use pass simultaneously and asserts exactly one grant. Stress behaviour was measured with up to 16 concurrent clients (Chapter 8), with no failed request.

9.5 Test Case Specification

Table 9.1: Representative test cases (actual result = observed in the run above)

TC	Req.	Scenario and input	Expected = actual	Status	Evidence (test)
TC-01	FR-01	wrong password; 11 rapid attempts	401; 11th returns 429	Pass	`test_bad_password_fails`, `test_login_is_rate_limited`
TC-02	FR-02	each role opens another role's portal	redirected / 403	Pass	`test_*_cannot_access_other_dashboards`
TC-03	FR-03	list and filter patients by tier and name	filtered roster	Pass	`test_list_patients_filtered_by_risk_tier`
TC-04	FR-04	live prediction for a patient	tier, score, 4 votes	Pass	`test_predict_patient_risk_live`, `test_each_model_reports_its_tier_and_risk_score`
TC-05	FR-04	data seeded 10 days earlier, 72-h chart	4 of 5 readings returned	Pass	`test_vitals_window_ends_at_latest_reading_not_wall_clock`
TC-06	FR-05	acknowledge alert with a forged name	signed-in name recorded	Pass	`test_audit_log_records_the_signed_in_clinician_not_the_typed_name`
TC-07	FR-06	upload record with invalid category	422	Pass	`test_vault_upload_invalid_category`
TC-08	FR-07	lifetime 10^{12} min, uses 10^{9}	422	Pass	`test_pass_lifetime_and_quota_are_bounded`
TC-09	FR-08	forged, expired, revoked tokens	denied and logged	Pass	`test_forged_and_tampered_token_rejection`, `test_expired_token_rejection`
TC-10	FR-08	4 concurrent scans of a single-use pass	exactly 1 grant	Pass	`test_single_use_pass_is_granted_once_under_concurrent_scans`
TC-11	FR-08	pass scoped to allergies	only allowed records	Pass	`test_scope_isolation_and_zero_data_leakage`

TC	Req.	Scenario	Expected = actual	Status	Evidence
TC-12	FR-10	narrative request	no identifier in prompt	Pass	`test_patient_clinical_narrative_endpoint`, security tests
TC-13	FR-11	forbidden or invalid what-if features	422 with code	Pass	`test_research_prediction_rejects_forbidden_and_invalid_inputs`
TC-14	FR-12	saved AUROC or fold fingerprint altered	503, no stays served	Pass	`test_research_stays_fail_closed_when_scores_do_not_reproduce_the_verified_run`
TC-15	FR-11	tampered model file	never deserialised	Pass	`test_tampered_model_is_never_deserialized`
TC-16	NFR	research mode with `DATA_MODE=mimic` under `wsgi.py`	refuses to start	Pass	`test_wsgi_entrypoint_refuses_research_mode`

9.6 Defect Identification and Resolution

A code audit of the backend and frontend during development found defects that were fixed and retested (all fixes have regression tests):

Table 9.2: Significant defects found and resolved

Defect	Cause	Resolution and retest
Single-use pass redeemable several times	check-then-update race	atomic conditional `UPDATE`; TC-10
Failed prediction shown as “Low risk, 85 %”	UI defaults masked errors	explicit “Prediction unavailable”; UI review
Audit log accepted typed names	identity from request body	identity from session; TC-06
Plain-text demo passwords in source	legacy design	scrypt hashes, production overrides; credential tests
500 errors on malformed bodies	unchecked JSON types	object-only payloads, bounds; parametrised tests
Empty 72-hour vitals chart	window anchored to wall clock	anchored to latest reading; TC-05
Decision tree output exactly 0 % (research)	pure leaves with few deaths	m-estimate smoothing; unit test

9.7 Validation Against KPIs

Table 9.3: KPI targets (Table 1.1) against measured values

KPI	Target	Measured	Verdict
Functional correctness	100 %	354/354 tests	met
Prediction latency (1 client, p50)	$\leq$ 150 ms	71.9 ms	met
Reliability (16 clients)	0 failures	0 of 3,200	met
Throughput, vitals (16 clients)	$\geq$ 100 req/s	268.8 req/s	met
Memory under load	$\leq$ 512 MB	206.1 MB	met
Synthetic hold-out accuracy	$\geq$ 0.91	1.00 (20 patients)	met, but see Section 10.4.2
Research evaluation with CI	reported	AUROC with 95 % CI	met (as reporting); performance not established

9.8 Limitations and Untested Scenarios

No automated browser end-to-end suite exists (the user interface was exercised manually and through the scripted screenshot run in Chapter 10); no cross-browser, accessibility-tool or penetration test was performed; load was tested only to 16 local clients; the Docker image could not be built on the development machine (Docker not installed); the narrative path was tested with a stubbed Gemini API; and model calibration was not evaluated.

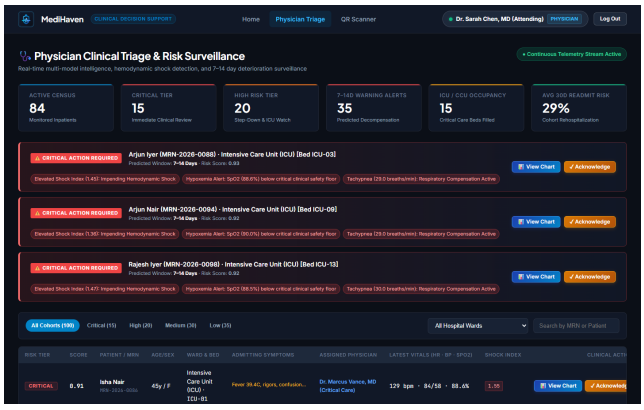
10 Results, Demonstration and Discussion

10.1 Implemented Features

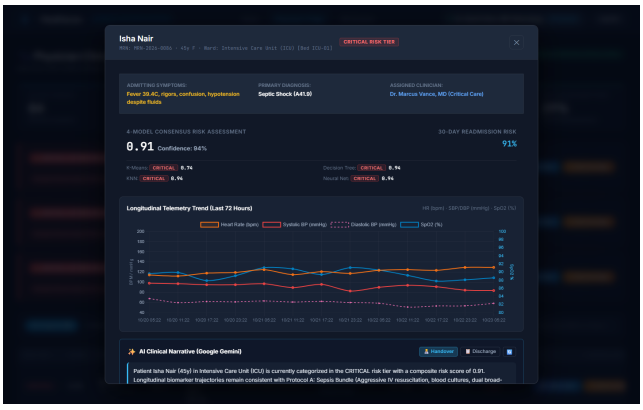
All objectives O1–O7 are implemented: role-based portals over 44 routes; the four-model synthetic ensemble with per-model votes and explanations; the patient vault with scoped, expiring, use-limited QR passes and an audit log; optional de-identified Gemini narratives; the MIMIC-IV Demo task with grouped cross-validation; loopback-only research serving with verified artifacts, a model-analytics page, a what-if estimator and a stay-level research triage page; and production hardening.

10.2 Screenshots and Demonstration Evidence

The screenshots below were captured on 26 October 2026 from the running application with an automated browser (Microsoft Edge driven by Playwright). Synthetic pages use a copy of the synthetic demo database; research pages show aggregate results only (the research triage page is cropped to its summary cards).



(a) Triage dashboard: KPIs, alert feed, roster

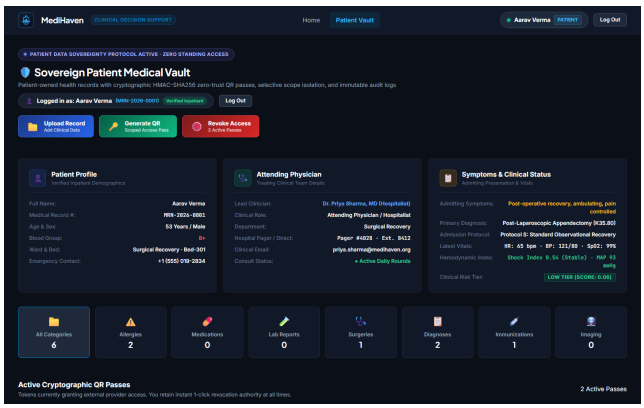


(b) Patient chart: 4 model votes and 72-h vitals

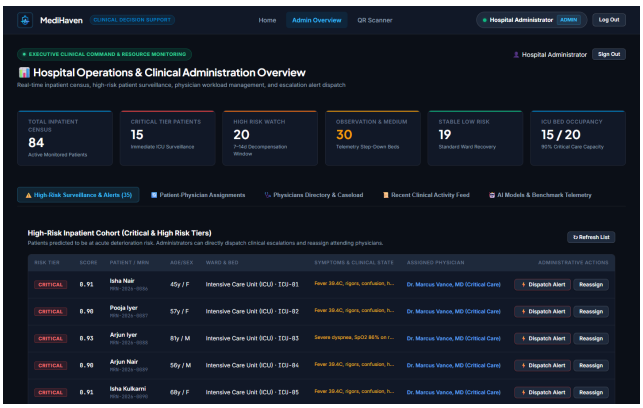
Figure 10.1: Physician portal on the synthetic cohort

10.3 Functional Results

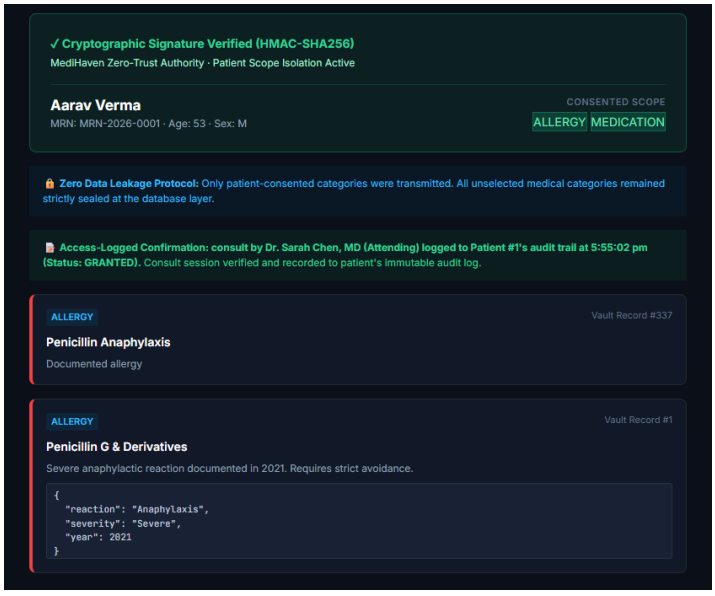
Each functional requirement is met by at least one passing test (Table 9.1). In the synthetic demo every one of the 100 patients returns a tier, a score, four model votes and a non-empty 72-hour vitals series; a single-use pass grants exactly one access even under concurrent scans; research mode refuses unsafe configuration and tampered artifacts.



(a) Patient vault



(b) Administrator overview



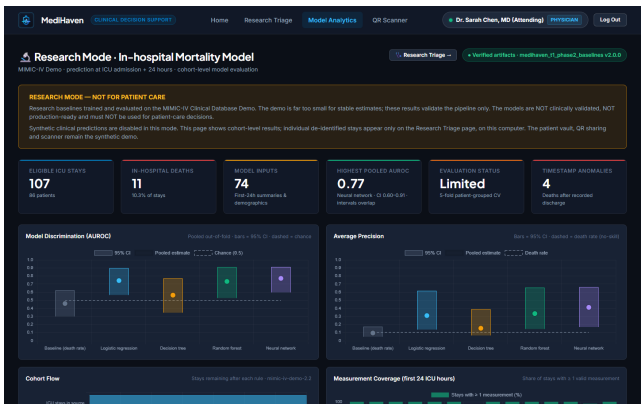
(c) QR scanner result: only the consented categories, access logged under the signed-in clinician

Figure 10.2: Patient, administrator and scanner portals (synthetic data)

10.4 Performance Results

10.4.1 Server performance

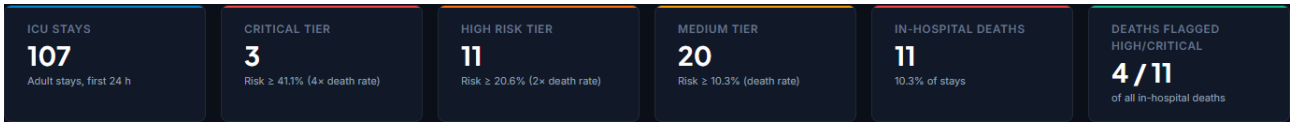
Measured server performance is reported in Chapter 8: median prediction latency 71.9 ms for one client, about 15 predictions/s at saturation, 268.8 simple reads/s at 16 clients, 0 errors and at most 206 MB of memory.



(a) Model analytics (cohort level)



(b) What-if estimate for an illustrative input



(c) Research triage summary cards

Figure 10.3: Research mode on the MIMIC-IV Demo (aggregate views only)

10.4.2 Synthetic ensemble benchmark

The project benchmark (`models_store/benchmark_report.json`, 23 October 2026) reports accuracy, precision, recall and F1 of 1.00 on a 20-patient hold-out split and in 5-fold cross-validation, with a mean inference latency of 24.4 ms (p95 29.5 ms). **These figures do not indicate clinical accuracy.** The synthetic patients are generated tier by tier from tier-specific value ranges, so the tiers are nearly separable by construction; perfect scores show only that the pipeline learns the generator.

10.4.3 Research task on the MIMIC-IV Demo

Cohort. Table 10.1 traces the cohort (dataset v1.2.0, task v3). "Eligible" means the first ICU stay of an adult admission, still alive, still in the ICU and still in hospital at the index time (ICU admission + 24 h). The label is in-hospital death after the index time; features use only the first 24 ICU hours. All 11 positive stays, including the 4 with a death time recorded after the discharge time, are in the primary evaluation cohort of 107 stays; sensitivity analysis A re-scores the primary predictions without the 4 anomalies, and B retrains without them (103 stays, 7 deaths).

Baselines. Table 10.2 gives the pooled out-of-fold results of model set v2.0.0 (5-fold StratifiedGroupKFold by patient, seed 42; 95 % subject-bootstrap intervals; threshold = training prevalence). Figure 10.4 shows the AUROC intervals.

Sensitivity analyses (103 stays, 7 deaths). A (anomalies removed from scoring only): AUROC LR 0.73, DT 0.52, RF 0.73, MLP 0.78. B (retrained without anomalies, folds with 1–2 deaths): LR 0.40 [0.08–0.76], DT 0.61, RF 0.71, MLP 0.39 [0.02–0.78]. **Stay-level view.** With tiers at fixed multiples of the death rate (Medium $\geq 1\times$, High $\geq 2\times$, Critical $\geq 4\times$

Table 10.1: Research cohort flow (MIMIC-IV Clinical Database Demo 2.2)

Step	Stays
ICU stays in source	140
excluded: not the first ICU stay of the admission	−12
excluded: died before the index time	−1
excluded: left the ICU before the index time	−20
excluded: ambiguous outcome fields	0
Eligible cohort (primary evaluation)	**107**
positive (in-hospital death): 7 consistent + 4 flagged timestamp anomalies	11 (10.3 %)
negative	96
Sensitivity cohort (anomalies removed)	103 (7 deaths)

Table 10.2: Research baselines, primary analysis (107 stays, 11 deaths; evaluation status: limited)

Model	AUROC [95% CI]	Avg. precision [95% CI]	Brier	Sens. / Spec.
Prior-only reference	0.46 [0.30–0.62]	0.10 [0.05–0.17]	0.092	1.00 / 0.00
Logistic regression	0.74 [0.57–0.90]	0.31 [0.14–0.62]	0.086	0.73 / 0.76
Decision tree (smoothed)	0.56 [0.35–0.77]	0.15 [0.07–0.39]	0.098	0.36 / 0.75
Random forest	0.73 [0.53–0.91]	0.34 [0.14–0.66]	0.082	0.64 / 0.62
Neural network (MLP)	0.77 [0.60–0.91]	0.41 [0.17–0.66]	0.087	0.55 / 0.85

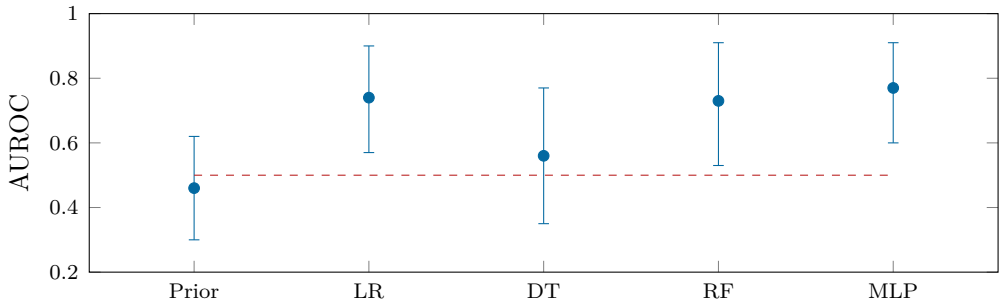


Figure 10.4: Pooled out-of-fold AUROC with 95% subject-bootstrap intervals (Table 10.2); dashed line: chance (0.5)

10.3 %), the 107 stays fall into 3 Critical, 11 High, 20 Medium and 73 Low, with 2, 2, 4 and 3 deaths respectively; 4 of the 11 deaths were placed in High or Critical.

10.5 Comparison with Existing Systems

Compared with the systems in Table 2.1, MediHaven adds patient-controlled sharing, explicit uncertainty and fail-closed research serving, but its discrimination on real data (AUROC about

0.73–0.77 with intervals spanning roughly 0.55–0.90) is far below large-cohort models such as [3] (0.93–0.94) and is not comparable with them, given 11 deaths and no external validation. Bedside scores remain the only clinically validated option of those compared.

10.6 Achievement of Objectives

Table 10.3: Objectives and supporting evidence

Objective	Evidence	Status
O1 portals and API	44 routes; RBAC tests; Figures 10.1–10.2	achieved
O2 explainable ensemble	votes, rule path, KNN cases; model tests; Figure 10.1(b)	achieved
O3 secure vault and QR	scope, expiry, atomic quota, audit; TC-08 to TC-11	achieved
O4 research task and evaluation	Tables 10.1–10.2	achieved (pipeline), performance not established
O5 verified research serving	hash verification, OOF reproduction; TC-14, TC-15	achieved
O6 data protection	security tests; Section 4.7	achieved for the prototype scope
O7 testing and measurement	354 passing tests; Chapter 8	achieved

10.7 Discussion of Findings

The contrast between perfect synthetic scores and modest, uncertain real-data scores is the central finding: model quality claims depend on the data, and synthetic benchmarks validate pipelines, not medicine. On the real cohort, the learned models beat the prior-only reference in the primary analysis, but their intervals overlap heavily, the rankings change between analyses,

and sensitivity analysis B shows that logistic regression and the MLP collapse when only 7 deaths remain. The four timestamp anomalies are 4 of 11 positives, so the labelling rule materially affects results. The smoothed decision tree trades discrimination for interpretability and never outputs 0 % or 100 %.

10.8 Practical Applicability

MediHaven is suitable for teaching, demonstrations and prototyping user interfaces and data-protection patterns. It must not be used for diagnosis, treatment or any other patient-care decision. Larger, multi-site, credentialed data, calibration analysis, prospective evaluation and regulatory review would be required before any clinical use could be considered.

11 Conclusion and Future Scope

11.1 Conclusion

MediHaven integrates explainable four-model triage on synthetic inpatients, a patient-controlled medical vault with signed, scoped and auditable QR passes, and an opt-in research mode that rebuilds an in-hospital-mortality task from the MIMIC-IV Clinical Database Demo and evaluates baselines with patient-grouped cross-validation. The system is layered, tested (354 passing tests), measured under load and hardened for deployment of the synthetic demo, while research mode stays on the researcher's machine.

11.2 Achievement of Objectives

All seven objectives were achieved as engineering deliverables (Table 10.3). Objective O4 was achieved as a validated pipeline; clinically meaningful model performance was *not* established and was never an achievable aim on a 107-stay demo cohort with 11 deaths.

11.3 Major Engineering Contributions

A fail-closed research path (secret, bind-address and artifact-hash checks; no external AI); out-of-fold stay-level scores that must reproduce the verified run before being served; explicit handling of outcome timestamp anomalies; an atomic, race-free pass quota; leakage controls on time windows and feature provenance; and measured performance with an identified CPU-bound prediction bottleneck.

11.4 Current Limitations

Synthetic data cannot show clinical validity; the real cohort is tiny and single-centre, results are unstable across sensitivity analyses, and calibration is unassessed; the design is single-instance; the vault stores document references, not files; no browser end-to-end or penetration tests exist; and the demo accounts must be replaced in any deployment.

11.5 Future Enhancements

The following are future work, not implemented features: evaluation on credentialed full MIMIC-IV with temporal or external validation and calibration analysis; per-patient feature caching and process-based inference workers; a shared store (e.g. PostgreSQL and Redis) for

multi-instance deployment; real, virus-scanned document uploads; browser end-to-end and accessibility tests; and an independent security review.

11.6 Lessons Learned

Leakage hides in timestamps; small cohorts demand intervals and sensitivity analyses rather than single numbers; synthetic perfection is a warning sign, not a result; security defects (races, trusted client input, plain-text secrets) are found by deliberate review and fixed with regression tests; and frequent, reviewed commits through pull requests kept a three-person team's work integrated.

Research Paper and Plagiarism Report

Research Paper

[No research paper has been submitted or accepted for this project. If a paper is written, insert it here with its plagiarism report and, if accepted, the acceptance receipt, as the template requires.]

Plagiarism Report

[The institutional plagiarism report for this document has not been supplied. Insert the genuine report here (for example as `figures/plagiarism_report.pdf` with `\includegraphics`) once it has been generated. No similarity score is stated or implied by this placeholder.]

Bibliography

[1] G. B. Smith, D. R. Prytherch, P. Meredith, P. E. Schmidt, and P. I. Featherstone, "The ability of the National Early Warning Score (NEWS) to discriminate patients at risk of early cardiac arrest, unanticipated intensive care unit admission, and death," *Resuscitation*, vol. 84, no. 4, pp. 465–470, 2013. [Online]. Available: https://doi.org/10.1016/j.resuscitation.2012.12.016

[2] J.-R. Le Gall, S. Lemeshow, and F. Saulnier, "A new simplified acute physiology score (SAPS II) based on a European/North American multicenter study," *JAMA*, vol. 270, no. 24, pp. 2957–2963, 1993. [Online]. Available: https://doi.org/10.1001/jama.1993.03510240069035

[3] A. Rajkomar, E. Oren, K. Chen *et al.*, "Scalable and accurate deep learning with electronic health records," *npj Digital Medicine*, vol. 1, p. 18, 2018. [Online]. Available: https://doi.org/10.1038/s41746-018-0029-1

[4] S. Purushotham, C. Meng, Z. Che, and Y. Liu, "Benchmarking deep learning models on large healthcare datasets," *Journal of Biomedical Informatics*, vol. 83, pp. 112–134, 2018. [Online]. Available: https://doi.org/10.1016/j.jbi.2018.04.007

[5] A. Johnson, L. Bulgarelli, T. Pollard, S. Horng, L. A. Celi, and R. Mark, "MIMIC-IV clinical database demo (version 2.2)," PhysioNet, 2023. [Online]. Available: https://doi.org/10.13026/dp1f-ex47

[6] A. E. W. Johnson, L. Bulgarelli, L. Shen *et al.*, "MIMIC-IV, a freely accessible electronic health record dataset," *Scientific Data*, vol. 10, no. 1, p. 1, 2023. [Online]. Available: https://doi.org/10.1038/s41597-022-01899-x

[7] A. E. W. Johnson, T. J. Pollard, and R. G. Mark, "Reproducibility in critical care: a mortality prediction case study," in *Proceedings of the 2nd Machine Learning for Healthcare Conference*, ser. Proceedings of Machine Learning Research, vol. 68. PMLR, 2017, pp. 361–376. [Online]. Available: https://proceedings.mlr.press/v68/johnson17a.html

[8] A. Wong, E. Otles, J. P. Donnelly *et al.*, "External validation of a widely implemented proprietary sepsis prediction model in hospitalized patients," *JAMA Internal Medicine*, vol. 181, no. 8, pp. 1065–1070, 2021. [Online]. Available: https://doi.org/10.1001/jamainternmed.2021.2626

[9] A. Azaria, A. Ekblaw, T. Vieira, and A. Lippman, "MedRec: Using blockchain for medical data access and permission management," in *2016 2nd International Conference on Open and Big Data (OBD)*. IEEE, 2016, pp. 25–30. [Online]. Available: https://doi.org/10.1109/OBD.2016.11

[10] M. Jones, J. Bradley, and N. Sakimura, "JSON web token (JWT)," Internet Engineering Task Force, Request for Comments 7519, 2015. [Online]. Available: https://doi.org/10.17487/RFC7519

[11] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," Internet Engineering Task Force, Request for Comments 2104, 1997. [Online]. Available: https://doi.org/10.17487/RFC2104

[12] F. Pedregosa, G. Varoquaux, A. Gramfort *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[13] H. Harutyunyan, H. Khachatrian, D. C. Kale, G. Ver Steeg, and A. Galstyan, "Multitask learning and benchmarking with clinical time series data," *Scientific Data*, vol. 6, no. 1, p. 96, 2019. [Online]. Available: https://doi.org/10.1038/s41597-019-0103-9

[14] D. Van Veen, C. Van Uden, L. Blankemeier *et al.*, "Adapted large language models can outperform medical experts in clinical text summarization," *Nature Medicine*, vol. 30, no. 4, pp. 1134–1142, 2024. [Online]. Available: https://doi.org/10.1038/s41591-024-02855-5

[15] K. Singhal, S. Azizi, T. Tu *et al.*, "Large language models encode clinical knowledge," *Nature*, vol. 620, no. 7972, pp. 172–180, 2023. [Online]. Available: https://doi.org/10.1038/s41586-023-06291-2

[16] A. L. Goldberger, L. A. N. Amaral, L. Glass *et al.*, "PhysioBank, PhysioToolkit, and PhysioNet: components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000. [Online]. Available: https://doi.org/10.1161/01.CIR.101.23.e215

[17] R. Pirracchio, M. L. Petersen, M. Carone, M. Resche Rigon, S. Chevret, and M. J. van der Laan, "Mortality prediction in intensive care units with the Super ICU Learner Algorithm (SICULA): a population-based study," *The Lancet Respiratory Medicine*, vol. 3, no. 1, pp. 42–52, 2015. [Online]. Available: https://doi.org/10.1016/S2213-2600(14)70239-5

[18] G. S. Collins, J. B. Reitsma, D. G. Altman, and K. G. M. Moons, "Transparent reporting of a multivariable prediction model for individual prognosis or diagnosis (TRIPOD): the TRIPOD statement," *BMJ*, vol. 350, p. g7594, 2015. [Online]. Available: https://doi.org/10.1136/bmj.g7594

[19] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012. [Online]. Available: https://doi.org/10.1145/2382577.2382579

[20] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015. [Online]. Available: https://doi.org/10.1371/journal.pone.0118432

[21] B. Van Calster, D. J. McLernon, M. van Smeden, L. Wynants, and E. W. Steyerberg, "Calibration: the Achilles heel of predictive analytics," *BMC Medicine*, vol. 17, no. 1, p. 230, 2019. [Online]. Available: https://doi.org/10.1186/s12916-019-1466-7

[22] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019. [Online]. Available: https://doi.org/10.1038/s42256-019-0048-x

[23] M. P. Sendak, M. Gao, N. Brajer, and S. Balu, "Presenting machine learning model information to clinical end users with model facts labels," *npj Digital Medicine*, vol. 3, p. 41, 2020. [Online]. Available: https://doi.org/10.1038/s41746-020-0253-3

A Reproducibility and API Summary

Commands (Windows PowerShell, repository root)

Install	`.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt`
Synthetic demo	`.\.venv\Scripts\python.exe run_demo.py`
Tests	`.\.venv\Scripts\python.exe -m pytest tests/ -q -p no:cacheprovider`
Research dataset	`python -m src.tasks.build_mimic_t1 -mimic-dir <demo> -output-dir <out>`
Research training	`python -m src.training.train_mimic_t1 -research-dir <out>`
Research mode	`$env:DATA_MODE="mimic"; $env:MIMIC_OUTPUT_DIR="<out>"; python run_demo.py`
Production	`waitress-serve -host=0.0.0.0 -port=8000 wsgi:app` (see `docs/deployment.md`)
Load test	`python appendices/perf_measure.py` (this package)

API routes (44)

Group	Routes
Pages	`/`, `/login`, `/logout`, `/dashboard`, `/vault`, `/scanner`, `/admin`, `/research`, `/favicon.ico`
Auth	`POST /api/auth/login`, `/api/auth/logout`, `GET /api/auth/me`, `GET /api/health`
Patients	`GET|POST /api/patients`, `GET /api/patients/<id>`, `/vitals`, `/labs`, `/predict`, `/similar`, `/clinical-narrative`, `POST /reassign`, `GET /api/patients/physicians`
Dashboard & alerts	`GET /api/dashboard/summary`, `GET /api/alerts`, `POST /api/alerts/<id>/acknowledge`, `POST /api/alerts/dispatch`, `GET /api/alerts/activity`
Admin	`GET /api/admin/models`, `POST /api/admin/gemini/synthesize`
Vault	`GET /api/vault/<id>`, `/access-log`, `/active-tokens`, `POST /api/vault/upload`, `/share`, `/<id>/share`, `/access`, `/revoke/<token>`
Research	`GET /api/research/status`, `/model-card`, `/dashboard`, `/stays`, `/stays/<n>`, `POST /api/research/predict`

NIL AI Content Declaration

PLACEHOLDER — GENUINE NIL AI CONTENT IMAGE REQUIRED

The official NIL AI content declaration/detection image was not present in the supplied materials (report guidelines, report template or project repository). It has deliberately **not** been recreated or simulated.

To complete this page, save the genuine image issued for this report as:

`figures/nil_ai_declaration.png`

and rebuild the report (`tectonic main.tex`). The image then replaces this notice automatically, and this page remains the last page of the report.